SOFTWARE DEVELOPMENT WITH PYTHON

Constraints, Keys and CRUD Operations

Lecture Material and Practical Workbook Guide — Modules 3 and 4

Day 2: SQL Fundamentals and Relational Database Design
Module 3 — Constraints and Keys
Module 4 — CRUD Operations with SQL
Running project: **Student Course Enrollment API** · Target: **PostgreSQL 18**

Prepared 19 October 2026 · psql and VS Code · Builds on Modules 1 and 2

Contents

1 Part 0 — Before you begin . . . 5
1.1 Prerequisites . . . 5
1.2 The project tree after this module . . . 5
1.3 How to run a SQL file — three ways, and when to use each . . . 5
2 Part 1 — The tables we are protecting . . . 6
3 Part 2 — The problem, in plain language . . . 8
4 Part 3 — Key: the value that identifies a row . . . 8
5 Part 4 — Constraint: a standing rule about what a row may contain . . . 9
5.1 Constraint vs validation — the distinction that matters most . . . 9
6 Part 5 — The four failure modes . . . 10
7 Part 6 — UNIQUE, CHECK, PRIMARY KEY, FOREIGN KEY . . . 11
7.1 `UNIQUE` — the same email, twice . . . 11
7.2 `CHECK` — values that are present but impossible . . . 12
7.2.1 The subtlety that trips every class: `CHECK` and `NULL` . . . 13
7.3 `PRIMARY KEY` — properly this time . . . 14
7.4 `FOREIGN KEY` and the `enrollments` table . . . 14
7.5 `ON DELETE` — what should happen to the children? . . . 17
7.6 What is a transaction? (read this before the rehearsal) . . . 17
8 Part 7 — The build script . . . 18
8.1 Two rules that make a script trustworthy . . . 18
8.2 `db/sql/02_constraints.sql` — the complete file . . . 19
9 Part 8 — The lab file . . . 20
9.1 Why a separate lab file . . . 20
9.2 `db/sql/lab/03_lab_constraints.sql` . . . 21
10 Part 9 — Workbook: work through these in order . . . 23
11 Part 10 — Verification checklist . . . 24
12 Part 11 — Troubleshooting: the five errors you will actually meet . . . 24
13 Part 12 — Instructor notes . . . 25
14 Part 13 — Submission . . . 26
15 Part 14 — Where this goes next . . . 26
16 Part 0 — Before you begin . . . 27
16.1 Prerequisites . . . 27
16.2 The project tree after this module . . . 28

16.3 Running files — the three ways, in one place . . . 28
17 Part 1 — DML, and what CRUD actually stands for . . . 29
17.1 Three categories of SQL you have already used . . . 29
17.2 CRUD, defined . . . 29
18 Part 2 — `INSERT`: creating records . . . 29
18.1 The single-row insert . . . 30
18.2 Inserting many rows at once . . . 30
18.3 Two forms worth knowing, both previewed . . . 31
19 Part 3 — `SELECT`: retrieving records . . . 32
19.1 The shape of every SELECT . . . 32
19.2 Choosing columns and rows . . . 32
19.3 `NULL`, and the operator that must not be used on it . . . 33
19.4 `ORDER BY` and `LIMIT` . . . 34
19.5 Pattern matching with `LIKE` . . . 34
19.6 `IN`, `BETWEEN`, `DISTINCT`, and counting . . . 35
20 Part 4 — `UPDATE`: changing existing records . . . 36
21 Part 5 — `DELETE`: removing records . . . 38
22 Part 6 — The consequences of incorrect updates and deletions . . . 39
22.1 Transactions are the seatbelt — and they are not optional . . . 39
22.2 The cascade is a deletion multiplier . . . 40
22.3 Constraints turn some mistakes into non-mistakes . . . 40
22.4 `RETURNING` makes the result visible . . . 40
22.5 `TRUNCATE` is a different tool entirely . . . 41
22.6 A summary of defences . . . 42
23 Part 7 — SQL CRUD vs API CRUD . . . 42
23.1 The mapping . . . 42
23.2 The five differences that matter . . . 43
23.3 One preview, one warning . . . 44
24 Part 8 — The build script, and the two rules . . . 44
25 Part 9 — Workbook part one: the build script . . . 47
26 Part 10 — The lab file . . . 47
27 Part 11 — Workbook part two: verification checklist . . . 51
28 Part 12 — Troubleshooting . . . 52
29 Part 13 — Instructor notes . . . 52

30 Part 14 — Submission 53
31 Part 15 — Where this goes next 54
Sources 54

Module 3

Constraints and Keys — Creating and Enforcing Database Structures

Course: Software Development with Python · Track: PostgreSQL 18, Student Course Enrollment API · Session length: 140 minutes (recommended 2–3 hours) · Format: lecture (Parts 1–6) + workbook (Parts 7–9)

How to use this module

This document is two things at once, and the distinction runs through every listing below.

The lecture (Parts 1–6) is the reading. It defines the concepts — key, constraint, referential integrity, transaction — before it uses them, and explains each statement line by line so you can teach it back. Read it in order.

The workbook (Parts 7–9) is the doing. It names the exact files to create, the exact file to paste into, the exact command to run, and the exact output to expect. Work through it with the database open beside you.

Every code listing in this module carries a destination marker, and you should follow it exactly:

Marker	What it means
`APPEND TO db/sql/02_constraints.sql`	Part of the build script. Paste it in the file at that position. This script must always run clean.
`LAB FILE --- db/sql/lab/03_lab_constraints.sql`	An experiment, including experiments designed to fail. Paste it there, never in the build script.
`TYPE AT THE PROMPT`	A one-off read-only check. Type it into `psql` to look at something. Do not save it anywhere.

The rule behind the markers: **the build script contains no deliberate mistakes.** If a failing `INSERT` lived in `02_constraints.sql`, the script would stop dead on that error every time anyone ran it — which destroys the property we spent Module 2 building, that a script can be run twice with silence the second time. Experiments live in the lab file. This separation is not bureaucracy; it is the difference between a script a team can trust and a script nobody dares re-run.

Part 0 — Before you begin

Prerequisites

This module assumes three things. If any is not true, stop and fix it first.

1. **You can open `psql` and connect to your course database.** In a terminal run `psql -U postgres -d school_db`. You should land at a prompt like `school_db=#`. (A prompt ending `-#` just means psql is ready for input; `=>` in some versions is equally fine.)
2. **Modules 1 and 2 are done**, so `students` and `courses` exist. Check by listing them:

Listing 1. Check that Module 2's tables exist

```
\dt
```

   You should see both tables listed. If you see nothing or an error, go back to Module 2 first.

3. **Your project folder is open in VS Code**, with `db/sql/01_create_tables.sql` from Module 2 in it.

The project tree after this module

Listing 2. The project folder at the end of Module 3

```
student-enrollment-api/├──
 db/│
   ├── sql/│
   │   ├── 01_create_tables.sql        ← Module 2, already exists│
   │   ├── 02_constraints.sql          ← YOU BUILD THIS (Part 7)│
   │   └── 03_crud_queries.sql         ← Module 4, not yet│
   └── lab/│
       └── 03_lab_constraints.sql      ← YOU BUILD THIS (Part 8)└──
 (your FastAPI app lives here later in the course)
```

Create the `lab` folder now, in VS Code: right-click `db` → **New Folder** → name it `lab`. Then right-click `lab` → **New File** → `03_lab_constraints.sql`. Part 8 tells you exactly what goes in it.

How to run a SQL file — three ways, and when to use each

Way 1 — from inside `psql`, best while learning

You are at the `school_db=#` prompt and you type:

```
\i db/sql/02_constraints.sql
```

`\i` (or `\include`) means "run the contents of this file." The path is relative to **the directory you launched `psql` from**, not to where the file lives — this trips up almost everyone, so if `\i` says `could not open file`, check what folder your

terminal is in. On Windows, always use forward slashes, never backslashes, and wrap the path in quotes if it contains spaces:

```
\i 'C:/Users/you/student-enrollment-api/db/sql/02_constraints.sql'
```

Way 2 — from the terminal, best once files are settled

Quit `psql` or open a second terminal tab and run:

```
psql -U postgres -d school_db -f db/sql/02_constraints.sql
```

Here the path is relative to your terminal's folder, and the file must be saved before you run it. This is the form to put in a README or a Makefile, because it is one command, scriptable, and repeatable.

Way 3 — paste at the prompt

Type or paste statements directly into `psql`. Use it for quick experiments and read-only checks. Never use it for anything you want to keep: once you close the session, the typing is gone.

A fourth habit worth forming now: **save before you run.** `psql` reads the file from disk at the moment you invoke `\i`. If you pasted into VS Code but forgot Ctrl+S, you will run yesterday's file and conclude the server is broken.

Part 1 — The tables we are protecting

Module 2 built two tables. Read them once more with fresh eyes, because everything today refers back to these columns. The two listings below are reference, not code to type — but do run `\d students` and `\d courses` to see them live in your own database.

Listing 3. REFERENCE — the students table built in Module 2

```
CREATE TABLE students (
    id            integer GENERATED BY DEFAULT AS IDENTITY,
    full_name     text NOT NULL,
    email         text NOT NULL,
    date_of_birth date,
    created_at    timestamptz NOT NULL DEFAULT now(),
    PRIMARY KEY (id)
);
```

Line by line — students

```
id integer GENERATED BY DEFAULT AS IDENTITY
```

the row's number. `integer` is a whole number; the rest means the server hands out the next number automatically, so you never type an `id`.

`full_name text NOT NULL`

the student's name. `text` is any length of letters; `NOT NULL` means always required.

`email text NOT NULL`

the email. Required, but **not yet unique** — right now two students may share one. Making it unique is today's first job.

`date_of_birth date`

a calendar day, no time. No `NOT NULL`, so it is optional.

`created_at timestamptz NOT NULL DEFAULT now()`

the exact moment the row was created. `timestamptz` stores a real point in time with time zone; `DEFAULT now()` means the server fills it in.

`PRIMARY KEY (id)`

`id` is what identifies each row.

Listing 4. REFERENCE — the courses table built in Module 2

```
CREATE TABLE courses (
    id          integer GENERATED BY DEFAULT AS IDENTITY,
    course_code text NOT NULL,
    title       text NOT NULL,
    credits     integer NOT NULL,
    course_fee  numeric(8,2) NOT NULL,
    created_at  timestamptz NOT NULL DEFAULT now(),
    PRIMARY KEY (id)
);
```

Line by line — courses

`course_code text NOT NULL`

the short code, like `CS101`.

`title text NOT NULL`

the course's full name.

`credits integer NOT NULL`

how many credit units it is worth. Required — but **no rule yet says it must be positive**, which is exactly the hole we close today.

`course_fee numeric(8,2) NOT NULL`

the fee as money. `numeric(8,2)` stores an exact decimal: up to 8 digits total, 2 after the point, so `15000.50` fits exactly. Required.

`id`, `created_at` and `PRIMARY KEY` work exactly as in `students`.

Two observations to hold on to

`email` is required but can still repeat, and **`credits` is required but can still be negative**. Those are today's two targets.

Part 2 — The problem, in plain language

You already know how to hold a list of students in Python:

Listing 5. A Python list happily holds the same email twice

```
students = [
    {"id": 1, "full_name": "Ada Okonkwo", "email": "ada@school.edu"},
    {"id": 2, "full_name": "Ada Okonkwo", "email": "ada@school.edu"},
]
```

Two rows, same email, and Python is happy — a list is just a sequence of values with no opinion about what they mean. To catch the duplicate you must write the check yourself (a loop, a `set()`, an `if`) and remember to run it every time the list changes. And it protects only that list, in that one program, at that one moment.

A database table is not a fancier list. The difference is that **the database promises to keep the table's rows true**, to whatever rules you give it, for as long as the table exists — and enforces them on every write, no matter who performs it. That promise, and the machinery behind it, is this module.

Part 3 — Key: the value that identifies a row

A **key** is a column (or set of columns) whose value identifies one row and no other. Every row in `students` is a different person; the key value is what lets you say *this exact row, not any of the others*.

- A **primary key** is *the* key of the table — the one the rest of the database uses to refer to its rows. Exactly one per table. As Section 6.3 shows, its value can never be null and never repeats.
- A **candidate key** is any column or set of columns that *could* serve as primary key because its values are unique and never null. A table usually has several; the designer chooses. In `students`, both `id` and `email` are candidate keys once email is unique. We choose `id` as primary and leave `email` merely unique.
- A **composite key** is a key made of more than one column, where the *combination*

identifies the row. We build one today in `enrollments`.

- A **surrogate key** is an artificial key added for convenience — a plain `id integer` that exists only to be short and easy to index, with no business meaning. Our `students.id` is one. Whether to add a surrogate key beside a natural key is a real design debate; this course's answer is yes for `students` and `courses`, and no for the `enrollments` junction table — Section 6.4 explains why that is the more honest design there.

Part 4 — Constraint: a standing rule about what a row may contain

A **constraint** is a rule, stored inside the table definition, that every row — old and new — must satisfy, checked by the server on every insert and every update, forever. Two kinds: a **column constraint** applies to one column and sits in its definition (`email text NOT NULL`), and a **table constraint** is written as its own named element and may involve one column or several (`CONSTRAINT uq_students_email UNIQUE (email)`).

Every constraint has a **name**. PostgreSQL invents one if you do not supply it (`students_pkey`), but that name appears in every error the constraint ever produces — and in the API layer later, you will translate those errors into HTTP responses. A name you chose (`uq_students_email`) tells you instantly, from the error alone, which rule fired. We name every constraint in this module.

Constraint vs validation — the distinction that matters most

This is the conceptual heart of the module. When your FastAPI receives a JSON body it will run **validation**: `isinstance(data["email"], str)`, `len(data["full_name"]) > 0`, in Python, in your application, before the application touches the database. That is validation. A **constraint** is the same kind of check, but it lives inside the table, is written once in SQL, and is evaluated by the server on every write from every writer.

	Validation	**Constraint**
Where it runs	Your application, in Python	The database server, in SQL
Who it protects	Your app, from bad input *it* receives	The data, from bad input from *anyone* — another script, another app, a `psql` session, a future teammate
On failure	Your code raises an exception / returns 422	The server rejects the statement; nothing is written
Why you still need it	You can return a friendly, field-level error to the user	You cannot trust every future writer to remember to validate first

Both layers are necessary and neither is redundant. **Validation is courtesy** — it produces a good error message for the human who typed the bad email. **A constraint is law** — it holds regardless of whether anyone was polite. The professional answer is never "constraints replace validation" or "validation replaces constraints", but *validation for good errors, constraints for truth*.

Part 5 — The four failure modes

Every constraint in this module exists to prevent exactly one of four ways data can be wrong. This table is the spine of the whole module — the roadmap's "duplicate, missing, or invalid data" maps onto it directly.

Failure	In plain language	The constraint that prevents it
Duplicate	The same real thing appears twice (two students, one email)	`PRIMARY KEY`, `UNIQUE`
Missing	A value the business requires is absent (`full_name` empty)	`NOT NULL`
Invalid	A value is present but impossible (`credits = -5`)	`CHECK`
Wrong link	A row points at something that does not exist	`FOREIGN KEY`

A fifth, `DEFAULT`, completes the picture because it is the only one that is not a prohibition: it is a claim about what value to supply when the writer supplies none. Every constraint decision is a question of *which failure am I closing?*

See it happen — TYPE AT THE PROMPT

> Type these three lines now and watch them succeed. They are the evidence for the whole module. **Do not save them anywhere** — Part 8 gives you the repeatable version.

Listing 6. TYPE AT THE PROMPT — deliberately bad data, accepted by the unconstrained schema

```
INSERT INTO students (full_name, email) VALUES ('Ada Okonkwo', 'ada@school.edu');
INSERT INTO students (full_name, email) VALUES ('Ada Okonkwo', 'ada@school.edu');
INSERT INTO courses (course_code, title, credits, course_fee)
VALUES ('CS101', 'Introduction to Programming', -5, -200.00);
```

Two students with one email, and a course with **negative five credits** and a **fee of negative two hundred** — all accepted, because the only rules now are "a name is required", "an email is required", and "credits and fee are present". This module is the reply to those three lines.

Part 6 — UNIQUE, CHECK, PRIMARY KEY, FOREIGN KEY

`UNIQUE` — the same email, twice

The business rule: **one email, one student**. Likewise **one course code, one course**.

APPEND TO `db/sql/02_constraints.sql`

> Create the file now if it does not exist; Part 7 shows the finished version in order.

Listing 7. APPEND TO `db/sql/02_constraints.sql` — make email and `course_code` unique

```
ALTER TABLE students
    ADD CONSTRAINT uq_students_email UNIQUE (email);

ALTER TABLE courses
    ADD CONSTRAINT uq_courses_code UNIQUE (course_code);
```

Line by line

`ALTER TABLE students`

change an existing table; nothing is dropped and no data moves, which is why `ALTER` is the right verb.

`ADD CONSTRAINT`

the instruction: add a standing rule to this table.

`uq_students_email`

the **name** you give it (`uq_` for unique, then table, then column), so future error messages read clearly.

`UNIQUE`

the **type** of rule.

`(email)`

the **column** it applies to.

Read the whole line as: *add a rule named uq_students_email, of type unique, over the email column.* The semicolon is required — this statement travels to the server; it is not a local shortcut.

What happens when you break it. From Part 5 you have two Ada rows with the same email, so `uq_students_email` cannot be added yet — the rule would not be true for data you already hold. Find the duplicate first. The simplest way, fine for a class-sized table:

TYPE AT THE PROMPT — a read-only check

Type it, read the output, do not save it.

Listing 8. TYPE AT THE PROMPT — sort and eyeball for duplicates

```
SELECT email FROM students ORDER BY email;
```

Word by word: `SELECT email` — show me the email column. `FROM students` — from this table. `ORDER BY email` — sort alphabetically by email. Because sorting puts identical emails next to each other, **any duplicate appears as two identical lines side by side** — scroll and look for a pair that repeats.

When the table is too large to eyeball, the automatic version:

TYPE AT THE PROMPT — the scalable duplicate-finder

Note each word; this is the shape you will use in real reporting.

Listing 9. TYPE AT THE PROMPT — GROUP BY / HAVING find duplicates automatically

```
SELECT email, count(*) FROM students
GROUP BY email
HAVING count(*) > 1;
```

Word by word. `SELECT email, count(*)` — show each email and a count of how many rows are in its group; `count(*)` means "count all the rows in the group." `GROUP BY email` — gather every row sharing an email into one group. `HAVING count(*) > 1` — keep only groups whose count exceeds one. The result lists exactly the duplicated emails.

Either way, once you know which emails repeat, **you** decide which row is real and delete or merge the other — that is a judgement about your data, not something the database should guess. Delete the duplicate Ada, then run `02_constraints.sql`.

The lesson to state out loud

`ADD CONSTRAINT` checks the rows you already have. A constraint is not a promise about future data; it is a judgement about all data, old and new.

`CHECK` — values that are present but impossible

`NOT NULL` asks "is it there?". `CHECK` asks "is it *sensible*?" A `CHECK` holds a boolean expression, and every row must make it true.

APPEND TO `db/sql/02_constraints.sql`

Paste below the `UNIQUE` block.

Listing 10. APPEND TO `db/sql/02_constraints.sql` — make the courses table sensible

```
ALTER TABLE courses
    ADD CONSTRAINT ck_courses_credits CHECK (credits > 0);
```

```
ALTER TABLE courses
    ADD CONSTRAINT ck_courses_fee CHECK (course_fee >= 0);

ALTER TABLE students
    ADD CONSTRAINT ck_students_dob CHECK (date_of_birth <= CURRENT_DATE);
```

Line by line

`CONSTRAINT ck_courses_credits CHECK (credits > 0)`

The name comes first (`ck_` for check, then table and column) so the error message will be readable. `CHECK` introduces the rule, and `(credits > 0)` is evaluated **for every row, on every insert and every update** — not once when you add it, but from this moment onward, forever.

`CHECK (course_fee >= 0)`

Note `>=` ("greater than or equal to") rather than `>`: a **free** course is a legitimate thing for a school to run. This is a modelling decision, not a syntax preference; the rule encodes the business, so the business gets a vote.

`CHECK (date_of_birth <= CURRENT_DATE)`

This one has no column on the left of the comparison; it compares the stored birth date against the **current date, evaluated by the server at the moment of the write**. A student born next year cannot be inserted.

`CURRENT_DATE`

A keyword, not a function call — hence no parentheses.

The failure, made repeatable — see Part 8, Task E, where this exact insert is kept in the lab file so you can re-run it whenever you like.

Listing 11. The error the server returns, verbatim

```
ERROR:  new row for relation "courses" violates check constraint "ck_courses_credits"
DETAIL:  Failing row contains (-5, -200.00).
```

The transaction is abandoned; nothing is written. Notice the `DETAIL`: when a `CHECK` fails, PostgreSQL shows you the offending values. That is the answer to "what went wrong", handed to you.

7.2.1 The subtlety that trips every class: `CHECK` and `NULL`

Read this carefully; it appears in real bug reports. A `CHECK` is satisfied if its expression is **true, or `NULL`** (unknown). It is violated only when the expression is **false**.

So if a column is nullable and holds `NULL`, this:

Listing 12. This constraint does NOT fire on NULL

```
CHECK (credits > 0)
```

…**passes**. `NULL > 0` is not true; it is *unknown*, and unknown is not false, so the row is admitted. If "credits must exceed zero" is a rule you mean absolutely, you need both rules together:

Listing 13. NOT NULL and CHECK together close both doors

```
credits integer NOT NULL,
CONSTRAINT ck_courses_credits CHECK (credits > 0)
```

`NOT NULL` says it must be there; `CHECK` says what is there must be greater than zero. Together they close both doors. This is exactly why the Module 2 schema declares `credits integer NOT NULL` rather than leaving it optional — **the two constraints are partners, and one alone is half a promise.**

`PRIMARY KEY` — properly this time

Module 2 wrote `PRIMARY KEY (id)` and called it a preview. The full meaning: a primary key is **the identity of a row** — the one thing that says *this row is this row and no other*. In PostgreSQL it grants three guarantees at once:

1. **Uniqueness** — no two rows share the key value (a primary key creates a unique index automatically).
2. **Presence** — the key column can never be `NULL`; PostgreSQL adds an implicit `NOT NULL` even where you did not write one.
3. **Stable reference** — the key is the handle other tables use to point at this row, which is what makes it a valid target for the foreign keys below.

TYPE AT THE PROMPT — read the structure the server built for you

> Type `\d students`, then look at the `Indexes:` section: `students_pkey` and `uq_students_email` are both there. Every uniqueness promise comes with machinery to enforce it quickly. (Indexes are their own module; know only that for now.)

One honest aside: `GENERATED BY DEFAULT AS IDENTITY` means the server assigns the id *unless a writer explicitly supplies one* — deliberate, because loading a real class list from a CSV needs explicit ids while day-to-day API traffic must not be able to choose its own. `BY ALWAYS` refuses explicit ids unless the writer says `OVERRIDING SYSTEM VALUE`: stricter, occasionally useful, and not what this course writes.

`FOREIGN KEY` and the `enrollments` table

The relationship first. Our project has a **many-to-many** relationship: one student takes many courses, one course is taken by many students. A relational database cannot represent many-to-many directly in one table — a row cannot belong to many parents. The standard solution, used by every serious course and every ORM you

will ever meet, is the **junction table** (also called a link or bridge table): a table whose rows *are* the relationships. A row in `enrollments` describes no student; it describes **the act of one student taking one course** — and that act carries its own data (`enrolled_on`), a fact belonging to neither parent and having no home anywhere else.

APPEND TO `db/sql/02_constraints.sql` — but it goes last

> It goes after the `DROP TABLE IF EXISTS enrollments;` line at the very top. Part 7 shows the file in correct order.

Listing 14. APPEND TO `db/sql/02_constraints.sql` — the junction table

```
CREATE TABLE enrollments (
    student_id  integer      NOT NULL,
    course_id   integer      NOT NULL,
    enrolled_on timestamptz  NOT NULL DEFAULT now(),

    CONSTRAINT pk_enrollments
        PRIMARY KEY (student_id, course_id),

    CONSTRAINT fk_enrollments_student
        FOREIGN KEY (student_id)
        REFERENCES students (id)
        ON DELETE CASCADE,

    CONSTRAINT fk_enrollments_course
        FOREIGN KEY (course_id)
        REFERENCES courses (id)
        ON DELETE CASCADE
);
```

Line by line

`student_id integer NOT NULL,`

This column does not store student details; it stores a **reference** to a row in `students`. That is the whole idea of a foreign key: a column whose value means *the row in the other table whose id is this*. The type must match the referenced column exactly — `students.id` is an identity integer, so this is an integer, and a mismatch here is one of the most common errors students hit. `NOT NULL` because "an enrollment of nobody" is meaningless, and — per the `NULL` subsection above — it also closes the `NULL` loophole on the `CHECK`s.

`course_id integer NOT NULL,`

The second parent; a junction table has exactly as many foreign keys as the relationship has parents.

`enrolled_on timestamptz NOT NULL DEFAULT now(),`

The *relationship's* own data, answering "when did this student join this course?", a question only this table can answer. `DEFAULT now()` means the

server stamps it; `NOT NULL` keeps it paired with the other guarantees.

```
CONSTRAINT pk_enrollments PRIMARY KEY (student_id, course_id),
```

A **composite** primary key: uniqueness is enforced on the *pair*. Here is the arithmetic worth savouring: a composite primary key is a uniqueness rule over two columns, so this single line gives you, free, the rule "a student cannot enrol in the same course twice". You did not write a separate `UNIQUE`; you did not need to — the composite key *is* the uniqueness rule, and it is also the primary key. It also expresses identity honestly: what makes an enrollment unique? Not a single id — the *combination*. This is the moment to show students that the "one auto-increment id per table" reflex is not always right. To be fair to that reflex: many production systems *do* add a surrogate `id` to junction tables, usually so the key is a single small value easy to put in a URL. Both designs are legitimate; the composite key teaches the concept more honestly.

```
CONSTRAINT fk_enrollments_student FOREIGN KEY (student_id) REFERENCES students
(id) ON DELETE CASCADE,
```

Read as one sentence: *this column (`student_id`) is a foreign key; it must point at a row in `students`, specifically the row whose `id` matches; and if that parent row is ever deleted, delete me too.* `REFERENCES students (id)` is the **referential integrity** promise: every value in `student_id` must currently exist as an `id` in `students`, or the write is rejected.

`ON DELETE CASCADE` is the next subsection.

The promise, enforced.

TYPE AT THE PROMPT — a live demonstration

Run both lines now; the first succeeds, the second is refused.

Listing 15. TYPE AT THE PROMPT — the first insert succeeds, the second is refused

```
INSERT INTO enrollments (student_id, course_id) VALUES (1, 1);
INSERT INTO enrollments (student_id, course_id) VALUES (999, 1);
```

Listing 16. The error the server returns, verbatim

```
ERROR:  insert or update on table "enrollments" violates foreign key constraint "
fk_enrollments_student"
DETAIL:  Key (student_id)=(999) is not present in table "students".
```

That is the sentence the roadmap calls *referential integrity*: an enrollment cannot point at a student who does not exist — not "should not", **cannot**. The check ran in the same transaction as the insert, on the server, before the row was written. Now try enrolling the same pair again, student 1 course 1, and the composite primary key refuses it, exactly as predicted.

A requirement to know before you need it. A foreign key can only point at a column that is itself **unique** — a `PRIMARY KEY` or `UNIQUE`. If you reference a plain column, PostgreSQL says `ERROR: there is no unique constraint matching given keys` (SQLSTATE 42830). This is not arbitrary; it is *logical*: if two students shared id 7, then "the enrollment pointing at 7" would be ambiguous — which student? Uniqueness in the parent is what makes the reference mean exactly one thing.

`ON DELETE` — what should happen to the children?

Here is a genuine design question, and the fact that the database asks it is one of the best arguments for constraints. When a parent row is deleted, what should happen to the rows pointing at it? PostgreSQL gives five behaviours; the right one depends on the business, not on the syntax.

Behaviour	What it does	When it is right
`NO ACTION` (default)	The delete is **refused** while children exist	"You cannot delete a course students are enrolled in" — force a human to decide
`RESTRICT`	Like `NO ACTION`, checked immediately rather than at end of statement	Rarely needed; a stricter cousin of the default
`CASCADE`	The child rows are **deleted too**	A student's enrollments are meaningless without the student
`SET NULL`	The child's reference becomes `NULL`	A document should survive the deletion of what it referenced
`SET DEFAULT`	The child's reference resets to a default	Rare; listed for completeness

Our project uses `ON DELETE CASCADE` on both parents, and that is a real business decision: **an enrollment without a student or without a course is not a fact about anything — it is a dangling sentence.** So when a student leaves, their enrollment rows go with them. `SET NULL` would be wrong here by design: we declared `student_id NOT NULL`, and `SET NULL` on a `NOT NULL` column is rejected — another demonstration of constraints working together. (You would use `SET NULL` on something like `approved_by_user_id` in an approval table, where "approved, by a user who no longer exists" should read as "not currently approved by anyone".)

What is a transaction? (read this before the rehearsal)

You have not formally met the word, so: a **transaction** is a group of one or more SQL statements the database treats as a single unit — either all of them happen, or none of them do. You open one with `BEGIN;` and close it with `COMMIT;` (make it permanent) or `ROLLBACK;` (undo everything since the `BEGIN`). Between those points your changes are real but temporary. This is exactly why we rehearse dangerous statements inside a transaction: try the dangerous thing, look at the result, undo it with `ROLLBACK`, and nothing is lost.

TYPE AT THE PROMPT, all five lines together

> They belong to one transaction, so paste them as a block rather than line by line — if you run `BEGIN;` alone and walk away, you leave a transaction

> open. (You would notice: an open transaction shows as `idle in transaction` in `pg_stat_activity`, and the lab file in Part 8 has a safety line for exactly this.)

Listing 17. TYPE AT THE PROMPT — the cascade rehearsal, made safe

```
BEGIN;
DELETE FROM students WHERE id = 1;
SELECT count(*) FROM enrollments WHERE student_id = 1;
ROLLBACK;
SELECT count(*) FROM students WHERE id = 1;
```

Line by line

`BEGIN;`

open a transaction; nothing is permanent until `COMMIT`.

`DELETE FROM students WHERE id = 1;`

delete one student (`WHERE` is taught properly in Module 4; read it here simply as "only the row with id 1"). The cascade does its work silently.

`SELECT count(*) FROM enrollments WHERE student_id = 1;`

count that student's enrollments inside the transaction; you will see **zero**, because they were cascaded away.

`ROLLBACK;`

undo everything, including the cascade.

The final `SELECT count(*) FROM students WHERE id = 1;` proves the student is back, and implicitly so are their enrollments. This is the safety net from Module 2, now protecting real data instead of structure.

Part 7 — The build script

Two rules that make a script trustworthy

1. **A build script contains no deliberate mistakes.** Every statement in it succeeds, in order, every time. Experiments that are *meant* to fail live in the lab file (Part 8).
2. **A build script is re-runnable.** Run it twice; the second run must be silent. This is what lets a teammate — or you, next week — rebuild the database from nothing by typing one command.

Two things break rule 2 when adding constraints, and both need explicit handling:

- `CREATE TABLE` fails if the table exists → `DROP TABLE IF EXISTS` first, **dependents before parents** (`enrollments` points at both other tables, so it goes first).
- `ALTER TABLE ... ADD CONSTRAINT` fails if the constraint exists → `ALTER TABLE ... DROP CONSTRAINT IF EXISTS` the same constraint first, then add it back. The rhythm is *remove the old rule, then install the new one, every time.*

`db/sql/02_constraints.sql` — the complete file

APPEND TO `db/sql/02_constraints.sql`

> If you have been pasting as you read, this section is the tidy reference. If you have not started the file, create it now and paste the entire block below in one go. Then run it with `\i db/sql/02_constraints.sql`.

Listing 18. The complete build script for Module 3

```
-- 02_constraints.sql
-- Day 2 / Module 3: keys and constraints.
-- Run after 01_create_tables.sql. Safe to run any number of times.

-- dependents before parents: enrollments points at both other tables
DROP TABLE IF EXISTS enrollments;

-- make each ALTER re-runnable: remove the rule first, then add it back
ALTER TABLE students DROP CONSTRAINT IF EXISTS uq_students_email;
ALTER TABLE students DROP CONSTRAINT IF EXISTS ck_students_dob;
ALTER TABLE courses  DROP CONSTRAINT IF EXISTS uq_courses_code;
ALTER TABLE courses  DROP CONSTRAINT IF EXISTS ck_courses_credits;
ALTER TABLE courses  DROP CONSTRAINT IF EXISTS ck_courses_fee;

-- uniqueness: one email, one student; one code, one course
ALTER TABLE students
    ADD CONSTRAINT uq_students_email UNIQUE (email);

ALTER TABLE courses
    ADD CONSTRAINT uq_courses_code UNIQUE (course_code);

-- sanity: sensible values only
ALTER TABLE students
    ADD CONSTRAINT ck_students_dob CHECK (date_of_birth <= CURRENT_DATE);

ALTER TABLE courses
    ADD CONSTRAINT ck_courses_credits CHECK (credits > 0);

ALTER TABLE courses
    ADD CONSTRAINT ck_courses_fee CHECK (course_fee >= 0);

-- the junction table: one row = one student's place in one course
CREATE TABLE enrollments (
    student_id  integer      NOT NULL,
    course_id   integer      NOT NULL,
    enrolled_on timestamptz  NOT NULL DEFAULT now(),

    CONSTRAINT pk_enrollments
        PRIMARY KEY (student_id, course_id),
```

```
    CONSTRAINT fk_enrollments_student
        FOREIGN KEY (student_id)
        REFERENCES students (id)
        ON DELETE CASCADE,

    CONSTRAINT fk_enrollments_course
        FOREIGN KEY (course_id)
        REFERENCES courses (id)
        ON DELETE CASCADE
);
```

Line by line — the parts that are new today

`DROP TABLE IF EXISTS enrollments;`

first, because `enrollments` depends on both other tables; the dependency rule from Module 2 applies in full.

`IF EXISTS`

keeps the script re-runnable: on a fresh database there is nothing to drop, and that is not an error worth stopping for.

`ALTER TABLE students DROP CONSTRAINT IF EXISTS uq_students_email;`

`DROP CONSTRAINT` is the mirror of `ADD CONSTRAINT`, and `IF EXISTS` gives it the same forgiving behaviour.

Everything after is the repetition of the UNIQUE, CHECK and FOREIGN KEY sections above.

Then prove it. Run the file once (`\i db/sql/02_constraints.sql`), then run it a second time. If the second run is not silent, something in the file is not re-runnable and you have found a real defect the moment you went looking. Note: if you still have the two duplicate Ada rows from Part 5, the first run *will* fail on `uq_students_email` — which is correct behaviour, not a bug. Delete the duplicate first (Part 9, Task B).

Part 8 — The lab file

Why a separate lab file

Everything in Part 7 must succeed. But half of this module is about statements that *fail* — duplicate emails, negative credits, dangling references. Put those in the build script and the script dies on them every run. So they go here, where failing is the expected outcome and the failure is the lesson.

The lab file is **not** idempotent in the same way, and it does not need to be. Its tasks are numbered, each is self-contained, and each states what you should see.

db/sql/lab/03_lab_constraints.sql

APPEND TO `db/sql/lab/03_lab_constraints.sql`

> Create it in VS Code (Part 0.2) and paste the whole block below.

Listing 19. The complete lab file for Module 3

```
-- 03_lab_constraints.sql
-- Experiments for Day 2 / Module 3. Not part of the build.
-- How to use: uncomment ONE task, run it with \i db/sql/lab/03_lab_constraints.sql,
-- read the result, then comment it back out. Never run the whole file uncommented.

-- ============================================================
-- TASK A - prove UNIQUE stops a duplicate email
-- Expected: ERROR ... violates unique constraint "uq_students_email"
-- ============================================================
-- INSERT INTO students (full_name, email) VALUES ('Ada Okonkwo', 'ada@school.edu');

-- ============================================================
-- TASK B - find duplicates BEFORE a unique constraint can be added
-- Expected: one row per repeated email, and nothing if there are none
-- ============================================================
-- SELECT email, count(*) FROM students GROUP BY email HAVING count(*) > 1;

-- ============================================================
-- TASK C - sort and eyeball the duplicates
-- Expected: identical emails sitting next to each other
-- ============================================================
-- SELECT email FROM students ORDER BY email;

-- ============================================================
-- TASK D - CHECK rejects an impossible value
-- Expected: ERROR ... violates check constraint "ck_courses_credits"
-- ============================================================
-- INSERT INTO courses (course_code, title, credits, course_fee)
-- VALUES ('CS101', 'Introduction to Programming', -5, -200.00);

-- ============================================================
-- TASK E - a free course is allowed; a negative one is not
-- Expected: the first INSERT succeeds, the second is rejected
-- ============================================================
-- INSERT INTO courses (course_code, title, credits, course_fee)
-- VALUES ('CS200', 'Free Community Seminar', 1, 0.00);
-- INSERT INTO courses (course_code, title, credits, course_fee)
-- VALUES ('CS201', 'Bad Fee Course', 3, -50.00);

-- ============================================================
-- TASK F - CHECK passes on NULL: the loophole from section 6.2
-- Expected: the INSERT succeeds - the CHECK does not fire on unknown.
--           This is why NOT NULL and CHECK are partners.
-- Try it on a scratch table, not on courses:
-- ============================================================
-- CREATE TABLE scratch (credits integer CHECK (credits > 0));
-- INSERT INTO scratch (credits) VALUES (NULL);   -- accepted!
-- DROP TABLE scratch;

-- ============================================================
```

```
-- TASK G - FOREIGN KEY refuses a dangling reference
-- Expected: ERROR ... violates foreign key constraint "fk_enrollments_student"
-- ============================================================
-- INSERT INTO enrollments (student_id, course_id) VALUES (999, 1);

-- ============================================================
-- TASK H - the composite primary key forbids duplicate enrolment
-- Expected: ERROR ... violates unique constraint "pk_enrollments"
-- ============================================================
-- INSERT INTO enrollments (student_id, course_id) VALUES (1, 1);

-- ============================================================
-- TASK I - rehearse the cascade inside a transaction
-- Expected: 0 enrollments after the DELETE, student back after ROLLBACK.
-- SAFETY: if you are interrupted between BEGIN and ROLLBACK, type ROLLBACK;
-- immediately - psql will also warn you when you quit inside a transaction.
-- ============================================================
-- BEGIN;
-- DELETE FROM students WHERE id = 1;
-- SELECT count(*) FROM enrollments WHERE student_id = 1;
-- ROLLBACK;
-- SELECT count(*) FROM students WHERE id = 1;
```

Line by line, for the parts that are new

Every task begins with `-- ===` and a title, then a comment stating the **expected output** — read that line before you run the task, so you know what success looks like.

Each statement is commented with `--`. Uncommenting in VS Code means deleting the two dashes at the start of the line (place the cursor after `--` and press Backspace twice, or select the line and press Ctrl+/ which toggles the comment). Re-comment with Ctrl+/ when you are done.

Task F builds a throwaway `scratch` table on purpose, so that the demonstration of the `NULL` loophole cannot pollute your real `courses` table; it drops it again at the end.

Task I carries a **safety line** because it is the only task where an interruption can leave the database in a half-changed state.

Part 9 — Workbook: work through these in order

Keep `psql` open beside VS Code. For each task: open `db/sql/lab/03_lab_constraints.sql`, uncomment that task, save (Ctrl+S), switch to `psql`, run `\i db/sql/lab/03_lab_constraints.sql`, read the result, compare it to the stated expectation, then re-comment the task.

#	Task	Your job	Expected
A	`Run 02_constraints.sql`	Create and run the build script	Succeeds silently; running it a second time is silent too
B	`Lab Task B + C`	Find the duplicate Adas from Part 5 and delete one by id	The duplicate-finder lists the email; afterwards the count is gone
C	`Re-run A`	Now the `UNIQUE` constraint can be added	Succeeds silently
D	`Lab Task D`	Try to insert the negative-credits course	Rejected: `ck_courses_credits`
E	`Lab Task E`	Insert a free course, then a negative-fee course	First accepted, second rejected: `ck_courses_fee`
F	`Lab Task F`	Reproduce the `NULL` loophole on a scratch table	The `NULL` insert is **accepted** — and that surprises people, which is the point
G	`Lab Task G`	Enrol student 999	Rejected: `fk_enrollments_student`
H	`Lab Task H`	Enrol the same pair twice	Rejected: `pk_enrollments`
I	`Lab Task I`	Rehearse the cascade	0 enrollments inside the transaction; everything restored after `ROLLBACK`
J	`Verification (Part 10)`	Run the full checklist and record the output	Every row matches

If Task A fails on `uq_students_email`

You still have duplicate rows, which is the lesson from Section 6.1. Go to Task B.

If Task A fails on `DROP CONSTRAINT`

You are probably running `02_constraints.sql` before `01_create_tables.sql` exists, or you are connected to the wrong database — check the prompt says `school_db`.

If `\i` says "could not open file"

See Part 0.3 — the path is relative to where you launched `psql`, not where the file lives.

Part 10 — Verification checklist

Do not trust the DDL you wrote. Test the behaviour from the outside, the way your application will. **These are read-only checks:** run them and record the output, because you will submit it.

#	Check	Command	Expected
1	Rules exist on `students`	`\d students`	`students_pkey`, `uq_students_email`, `ck_students_dob`
2	Rules exist on `courses`	`\d courses`	`uq_courses_code`, `ck_courses_credits`, `ck_courses_fee`
3	Rules exist on `enrollments`	`\d enrollments`	`pk_enrollments`, both `fk_` constraints
4	FK targets are shown	`\d enrollments`, `Referenced by`	`students(id)` and `courses(id)`
5	Duplicate refused	`Lab Task A`	`uq_students_email`
6	Impossible value refused	`Lab Task D`	`ck_courses_credits`
7	Impossible date refused	`Insert date_of_birth in 2099`	`ck_students_dob`
8	Dangling link refused	`Lab Task G`	`fk_enrollments_student`
9	Duplicate enrolment refused	`Lab Task H`	`pk_enrollments`
10	Cascade rehearsed, not suffered	`Lab Task I`	Enrollments gone inside the transaction, all restored after
11	The script is honest	`Run 02_constraints.sql twice`	Second run silent

Part 11 — Troubleshooting: the five errors you will actually meet

SQLSTATE	What the server says	Why	What to do
23505	`duplicate key value violates unique constraint "..."`	A `PRIMARY KEY` or `UNIQUE` already holds that value	Find the existing row (`SELECT ··· WHERE email = ···`); decide which is real
23503	`violates foreign key constraint "···"` + `Key (···) is not present`	The referenced row does not exist	Insert the parent first, or fix the id you passed
23514	`new row for relation "···" violates check constraint "..."`	A `CHECK` expression evaluated false	Read the `DETAIL: Failing row contains (···)` line — the values are the answer
23502	`null value in column "···" violates not-null constraint`	A required column was left empty	Supply the value; `DEFAULT` only fires for columns that have one
42830	`there is no unique constraint matching given keys`	A `FOREIGN KEY` references a column that is not unique	Add `PRIMARY KEY`/`UNIQUE` to the referenced column, or reference the right one

A habit worth teaching explicitly: **when a constraint error appears in your logs, the constraint name is your first debugging step.** It names the rule that fired; `\d` on that table shows the rule; the `DETAIL` line shows the data that broke it. In the API layer later, these five codes are exactly what you translate into HTTP statuses — 409 for conflict, 400/422 for a bad request. The database is already classifying the problem for you.

Part 12 — Instructor notes

Timing (140 minutes). Parts 0–2 (anchor, annotated tables, the problem) ≈ 25 min. Parts 3–5 (key, constraint, validation vs constraint, four failure modes) ≈ 35 min — do not rush; the validation-vs-constraint distinction is the intellectual core. Sections 6.1–6.2 (`UNIQUE`, duplicate-finding, `CHECK`, the `NULL` subtlety) ≈ 30 min. Section 6.4 (`FOREIGN KEY`, `enrollments`) ≈ 35 min — the centrepiece; protect its time. Sections 6.5–6.7 (`ON DELETE`, transaction, the script) ≈ 10 min. Parts 9–11 (workbook, verification, troubleshooting) ≈ 15 min.

Live-demo order matters. Run the three bad inserts (Part 5) *before* the fixes, and read the error messages aloud. Students retain a constraint after watching the database refuse bad data far better than after watching it accept it silently.

Misconceptions to expect, and the one-line answer for each.

- *"Constraints are for the database, validation is for the app."* — Both. The app gives a good error message to the user; the database is the floor nothing gets through.
- *"`CHECK` rejects `NULL`."* — No. A `CHECK` passes unless the expression is definitively false; `NOT NULL` is a separate promise and the two work together (Lab Task F).
- *"A foreign key is just an integer column."* — It is an integer column *plus a standing instruction to the server*. Drop the constraint and it becomes an ordinary integer again.
- *"Every table needs a single auto-increment id."* — Every table needs *a* primary key. For a junction table the honest key is the pair.
- *"`UNIQUE` and `PRIMARY KEY` are the same."* — They share the uniqueness machinery; they differ in intent and in guarantees (a PK is also `NOT NULL`).
- *"A key is the id column."* — A key is any value or combination that identifies a row; in some tables the id is not the most meaningful key.
- *"Adding a constraint only affects future inserts."* — No. It is checked against the rows you already have too.

Discussion, if time allows (5 min). *"Should `email` be `PRIMARY KEY`, `UNIQUE`, or both?"* The answer — email identifies a person, but identity ids are what the API exposes; uniqueness is about the business rule, not identity — is exactly the reasoning a designer needs.

The workbook lesson worth naming explicitly. Task A failing because of leftover duplicate rows is not a nuisance; it is the single best demonstration in the module that constraints are judgements about *existing* data. If you have time, let a student hit that failure on purpose.

Do not teach today (name only, one sentence each): deferrable constraints

(`DEFERRABLE INITIALLY DEFERRED`, for bulk loads where children briefly reference rows not yet inserted), `NOT VALID` (add a foreign key to a huge existing table without scanning it), and triggers. Mention them so they are not a surprise later.

Version note. PostgreSQL 18 throughout. `NULLS NOT DISTINCT` on unique constraints (PostgreSQL 15+) makes a nullable column treat `NULL` as a value that must itself be unique: `CONSTRAINT uq_x UNIQUE NULLS NOT DISTINCT (col)`. Our `email` is `NOT NULL` so we do not need it, but students who know about it will not be surprised later.

Part 13 — Submission

1. **`db/sql/02_constraints.sql`** — committed to the project, every constraint named, proven re-runnable. Include the evidence: the command you ran twice and the silent second run.
2. **`db/sql/lab/03_lab_constraints.sql`** — with at least tasks A, D, G, H and I uncommented at least once each (re-comment them after; submit the file in its tidy state).
3. **`\d students`, `\d courses`, `\d enrollments`** — the three verification outputs showing every constraint by name.
4. **An error log** — one rejected statement per SQLSTATE: 23505 (unique), 23514 (check), 23503 (foreign key), and the duplicate-enrolment 23505 on `pk_enrollments`. Four screenshots or four copied terminal blocks.
5. **A written trace of the cascade rehearsal** (Lab Task I) — what each count showed at each step.
6. **Three written answers, one sentence each:** (a) why `CHECK (credits > 0)` alone is not enough; (b) why a foreign key needs a `UNIQUE` column in the parent; (c) what would go wrong in your API if the `enrollments` foreign keys were absent.

Part 14 — Where this goes next

The three tables are now structurally complete and rule-bound: `students`, `courses`, `enrollments`. They are disciplined, and mostly empty. **Module 4** puts data into them — `INSERT`, `SELECT`, `UPDATE`, `DELETE` — and examines the consequences of getting an `UPDATE` or `DELETE` wrong. The constraints built today are exactly what will make Module 4's dangerous statements safe to write: by the end of today, a wrong `DELETE` cannot orphan a row, because there are no orphans left to create.

Module 4

CRUD Operations with SQL — Inserting, Retrieving, Updating and Deleting Data

Course: Software Development with Python · Track: PostgreSQL 18, Student Course Enrollment API · Session length: 140 minutes (recommended 2–3 hours) · Format: lecture (Parts 1–8) + workbook (Parts 9–11) · Build file: `db/sql/03_crud_queries.sql` · Lab file: `db/sql/lab/04_lab_crud.sql`

How to use this module

Same contract as Module 3. **The lecture (Parts 1–8)** defines every concept before using it and walks each statement line by line. **The workbook (Parts 9–11)** names the exact files, the exact commands, and the exact output to expect. Every listing carries a destination marker.

Marker	What it means
`APPEND TO db/sql/03_crud_queries.sql`	Part of the build script (the seed data). Paste at that position. This script must always run clean.
`LAB FILE --- db/sql/lab/04_lab_crud.sql`	An experiment, including experiments designed to damage data. Never in the build script.
`TYPE AT THE PROMPT`	A one-off read-only check. Type into `psql`. Do not save it.

The reason for the split is sharper in this module than in the last one. Module 3's lab contained statements that *failed and stopped*. This module's lab contains statements that **succeed and destroy data** — a forgotten `WHERE` clause turns `UPDATE` or `DELETE` from "change one row" into "rewrite every row in the table", and you only find out afterwards. So: seeds and safe work in the build script; anything with a blast radius in the lab file, inside a transaction, with the rollback written before the damage.

Part 0 — Before you begin

Prerequisites

1. **`psql` open and connected to `school_db`.** Run `psql -U postgres -d school_db`; you should see the `school_db=#` prompt.
2. **Modules 1–3 done.** Confirm the schema is in place and constraint-bearing by running `\dt`. You should see `students`, `courses` and `enrollments`. If `enrollments` is missing, run Module 3's build script first: `\i db/sql/02_constraints.sql`.
3. **Module 3's script is proven re-runnable.** Run `\i db/sql/02_constraints.sql` once more, now. It must be silent. If it is not, fix it before starting today — you cannot trust a schema you cannot rebuild.
4. **Your tables are empty or nearly so.** Today's seed script starts by clearing them, so leftover experimental rows from Module 3's lab will not hurt. Say so out loud to your students anyway, because "the seed script wipes the tables" is

exactly the kind of sentence that must never be a surprise.

The project tree after this module

Listing 20. The project folder at the end of Module 4

```
student-enrollment-api/├──
 db/│
   ├── sql/│
   │   ├── 01_create_tables.sql        ← Module 2│
   │   ├── 02_constraints.sql          ← Module 3│
   │   └── 03_crud_queries.sql         ← YOU BUILD THIS (Part 9)│
   └── lab/│
       ├── 03_lab_constraints.sql      ← Module 3│
       └── 04_lab_crud.sql             ← YOU BUILD THIS (Part 10)
```

Create `db/sql/lab/04_lab_crud.sql` now: right-click `lab` → **New File** → `04_lab_crud.sql`. And create `db/sql/03_crud_queries.sql` next to `02_constraints.sql`.

Running files — the three ways, in one place

1 — inside `psql`, best while learning

Type `\i db/sql/03_crud_queries.sql`. The path is relative to **where you launched `psql`**, not where the file lives; on Windows use forward slashes and quote paths containing spaces.

2 — from the terminal, best once settled

`psql -U postgres -d school_db -f db/sql/03_crud_queries.sql`. Path relative to your terminal's folder; **save the file first** — `psql` reads from disk at the moment it runs.

3 — paste at the prompt

For quick experiments and read-only checks only. Anything you want to keep goes in a file.

Part 1 — DML, and what CRUD actually stands for

Three categories of SQL you have already used

Cat.	Full name	What it does	You have used
DDL	Data Definition Language	Defines structure	`CREATE TABLE`, `ALTER TABLE`, `DROP TABLE` (Modules 2–3)
DML	Data Manipulation Language	Works with the *rows* inside a structure	**Today: `INSERT`, `UPDATE`, `DELETE`**
DQL	Data Query Language	Reads rows without changing them	**Today: `SELECT`**

Strictly, `SELECT` is often lumped in with DML; separating it is worth the pedantry because **`SELECT` is the only one of the four that cannot destroy anything by accident.** That asymmetry is the entire safety argument of this module.

One more category you will meet in Day 3, named now so it is not a surprise: **TCL** — Transaction Control Language — which is just `BEGIN`, `COMMIT` and `ROLLBACK`.

CRUD, defined

CRUD is an acronym for the four operations every data system must support:

Letter	Operation	`SQL statement`	Modifies structure?	Modifies rows?
C	Create	`INSERT`	No	Yes — adds rows
R	Read	`SELECT`	No	No — never
U	Update	`UPDATE`	No	Yes — changes existing rows
D	Delete	`DELETE`	No	Yes — removes rows

(Read the "U" row carefully — it says `UPDATE`, not `DELETE`, and that is deliberate.)

The mistake is instructive: **Create and Delete sound like opposites, and in a filesystem they are, but in a database they are not.** `INSERT` adds a row *alongside* existing ones; `DELETE` removes one *while leaving the rest in place*. The true opposite of "add a row" is "change a row", which is `UPDATE`. So the pairs that actually mirror each other are `INSERT`↔`UPDATE` (writing) and `SELECT`↔`DELETE` — well, and not really, as Part 7 explains. Naming the operations precisely is the first step towards not reaching for the wrong one under pressure.

Part 2 — `INSERT`: creating records

TYPE AT THE PROMPT — a live demo of the three forms below

> Insert one student now; Part 9 will clear the tables anyway.

The single-row insert

Listing 21. TYPE AT THE PROMPT — the single-row insert

```
INSERT INTO students (full_name, email, date_of_birth)
VALUES ('Ada Okonkwo', 'ada@school.edu', '2003-04-15');
```

Line by line

`INSERT INTO students`

start a new row in this table.

`(full_name, email, date_of_birth)`

the columns you are supplying, in any order you like. Note `id` and `created_at` are absent: `id` is `GENERATED BY DEFAULT AS IDENTITY` and `created_at` is `DEFAULT now()`, so the server fills both without being asked.

`VALUES (...)`

the actual values, **positionally matched to the column list**: the first value goes in `full_name`, the second in `email`, the third in `date_of_birth`. Get the order wrong and the data lands in the wrong columns without complaint unless the types happen to differ — so always name your columns explicitly, even when you could omit the list.

`'2003-04-15'`

a string literal in single quotes; in PostgreSQL a `date` literal is written `'YYYY-MM-DD'` and the server converts it. Note the asymmetry students always trip on: **strings use single quotes, identifiers use double quotes.** `'ada@school.edu'` is a value; `"ada@school.edu"` would be a column name, and PostgreSQL would tell you the column does not exist.

Inserting many rows at once

TYPE AT THE PROMPT — paste this whole block

It is one statement producing three rows.

Listing 22. TYPE AT THE PROMPT — the multi-row insert

```
INSERT INTO students (full_name, email, date_of_birth) VALUES
    ('Tunde Balogun',   'tunde@school.edu',  '2002-09-30'),
    ('Chiamaka Nwosu',  'chiamaka@school.edu','2004-01-12'),
    ('Fatima Bello',    'fatima@school.edu',  '2001-07-04');
```

Line by line

```
VALUES ('Emeka Obi', ...), ('Ada Nwosu', ...), ...
```

The `VALUES` keyword is followed by several parenthesised tuples separated by commas, all in one statement.

The column list appears **once**, before the first tuple, and applies to all of them.

The trailing semicolon goes after the *last* tuple only.

Why prefer this over three separate statements? Three reasons, all practical: it is one round trip instead of three; if any row violates a constraint the whole statement fails, so you never get half a batch; and on a large table it is dramatically faster, because the server writes one contiguous block instead of repeatedly seeking.

Whenever you are loading rows from a file, this is the form you want.

Two forms worth knowing, both previewed

`INSERT ... SELECT` — copy rows from one table into another. You will meet it in Day 3 when joins make the source non-trivial; the shape is always the same:

Listing 23. PREVIEW — INSERT ... SELECT copies rows from a query

```
INSERT INTO enrollments (student_id, course_id)
SELECT id, 1 FROM students WHERE email = 'ada@school.edu';
```

`RETURNING` — ask the server to hand back what it just did. This is how you get at the `id` the server generated, without a second query and without a race:

Listing 24. TYPE AT THE PROMPT — RETURNING gives you the generated id

```
INSERT INTO students (full_name, email)
VALUES ('Emeka Obi', 'emeka@school.edu')
RETURNING id, created_at;
```

The reply is a small table — this is the row that now exists, with its server-assigned identity. In your FastAPI layer later, this is exactly what you want to send back to the client: the created object, as the database stored it. `RETURNING` also works on `UPDATE` and `DELETE`, and Part 6 uses it as a safety device.

The rule to take away: `INSERT` grows the table, and every row it writes is checked against every constraint from Module 3 — in the same transaction, before it exists.

Part 3 — SELECT: retrieving records

SELECT is the largest of the four statements and the one you will write most often, so it gets the most time.

TYPE AT THE PROMPT — run each listing in this part in order

> Everything in this section is read-only, so none of it can hurt you.

The shape of every SELECT

Listing 25. The clause order of every SELECT, always

```
SELECT  which columns
FROM    which table
WHERE   which rows (optional)
ORDER BY which order        (optional)
LIMIT   how many            (optional)
```

The clauses run in that order, always. Beginners memorise it as a fixed sentence, and that is a good instinct — SQL will reject a WHERE that appears after LIMIT, and the error message ("misplaced … clause") is unhelpful until you know the rule.

Choosing columns and rows

Listing 26. TYPE AT THE PROMPT — name the columns, filter with WHERE

```
SELECT full_name, email FROM students WHERE id = 1;
```

Line by line

`SELECT full_name, email`

name the columns you want.

`SELECT *`

the alternative, meaning "every column". It is fine for a quick look in psql, but it is a **professional habit to avoid in application code**, for two concrete reasons: if someone later adds a column (say a national ID number) your code silently starts receiving it, possibly sending it to a browser; and when a table has twenty columns, selecting six by name is clearer than scrolling past fourteen you do not use.

`FROM students`

the table.

`WHERE id = 1`

keep only rows where `id` equals 1. `WHERE` is the filter, and it is the clause that decides *which* rows; every danger in Part 4 comes from getting this clause wrong or forgetting it.

Comparison operators. The full set, worth having on one card: `=` equal, `<>` or `!=` not equal (both work; `<>` is standard SQL), `<` less than, `>` greater than, `<=` less than or equal, `>=` greater than or equal. For strings, comparison is **alphabetical**: `WHERE full_name > 'M'` gives every student whose name sorts after M, which is rarely what a beginner means but is exactly right for "show me everyone from N to Z".

Combining conditions uses `AND`, `OR` and `NOT`. `AND` requires both sides true; `OR` requires either.

Listing 27. TYPE AT THE PROMPT — AND combines two conditions; both must hold

```
SELECT full_name, email, credits FROM courses
WHERE credits > 2 AND course_fee < 50000.00;
```

Read it out loud as *"courses worth more than two credits **and** costing under fifty thousand"*. `OR` is where parentheses earn their keep — `WHERE credits > 4 OR course_fee = 0 AND credits = 1` is ambiguous to a human reader even though the database evaluates `AND` first, so write the parentheses you mean: `WHERE credits > 4 OR (course_fee = 0 AND credits = 1)`.

`NULL`, and the operator that must not be used on it

`NULL` means **no value at all** — not empty string, not zero, not false. It means the value is unknown or absent. This creates one rule that catches nearly every beginner in the first hour:

Listing 28. TYPE AT THE PROMPT — IS NULL is correct; = NULL returns nothing, silently

```
SELECT full_name, email, date_of_birth FROM students WHERE date_of_birth IS NULL;
```

Never write `= NULL`

It does not mean what it looks like. In SQL, comparing anything to `NULL` — even `NULL = NULL` — yields *unknown*, not true, so `WHERE date_of_birth = NULL` matches **zero rows, always**, and returns an empty result with no error at all. That silence is what makes it dangerous. You must use the dedicated forms: `IS NULL` and `IS NOT NULL`. The reason is worth understanding rather than memorising: `NULL` means "unknown", and a comparison against unknown can never be established as true, so it is not a row you can select.

There is a second `NULL` trap that will cost you an afternoon if nobody warns you: **`NOT` and `NULL` interact badly.** `WHERE credits <> 3` returns every row where credits is known *and* not 3 — it silently omits every row where credits is `NULL`, because `NULL <> 3` is unknown. If you want "everything except credits of 3, including the unknowns", you need the explicit three-way logic of `IS NULL OR <> 3`. This is not pedantry; it is the single most common source of "why is my report missing rows?"

`ORDER BY` and `LIMIT`

Listing 29. TYPE AT THE PROMPT — sort first, then limit

```
SELECT id, full_name, created_at FROM students
ORDER BY created_at DESC
LIMIT 3;
```

Line by line

`ORDER BY created_at DESC`

sort the rows by that column, **descending** (largest/newest first; the default with no `DESC` is ascending, oldest first).

`LIMIT 3`

return only the first three rows of that sorted result.

The two work together: `LIMIT` without `ORDER BY` returns three rows in whatever order the server happens to produce, which is **not guaranteed stable** and can differ between runs — a real bug source in reporting code. Always `ORDER BY` before you `LIMIT`, unless the order genuinely does not matter.

`ORDER BY credits DESC, title ASC`

You can order by more than one column: highest credits first, and within equal credits, alphabetical. This is how you get a stable, predictable report.

Pattern matching with `LIKE`

Listing 30. TYPE AT THE PROMPT — LIKE wildcards: % and _

```
SELECT id, full_name, email FROM students WHERE email LIKE '%@school.edu';
```

Line by line

`LIKE`

a pattern match on text, where two wildcards do the work.

`%`

means “any number of any characters, including none”. So `LIKE '%@school.edu'` matches any email *ending* in `@school.edu`, and `LIKE 'ada%'` matches names starting with “ada”.

`_`

means “exactly one character”. So `LIKE 'a_a'` matches a three-character name

starting with a and ending with a.

Read `%` as “anything” and _ as “one thing”.

A warning worth giving students: `LIKE` with a leading `%` cannot use an index, so it gets slower as the table grows — fine for a class-sized table, a real performance problem at scale. Also `LIKE` is **case-sensitive** in PostgreSQL. For case-insensitive matching, the modern answer is `ILIKE` (or `lower(email) LIKE lower('%@school.edu')`), which we meet in Day 3 with `EXPLAIN`.

`IN`, `BETWEEN`, `DISTINCT`, and counting

Listing 31. TYPE AT THE PROMPT — IN, BETWEEN, DISTINCT and count

```
SELECT id, full_name FROM students WHERE id IN (1, 3, 5);
SELECT title FROM courses WHERE credits BETWEEN 2 AND 4;
SELECT DISTINCT credits FROM courses ORDER BY credits;
SELECT count(*) AS total_students FROM students;
```

Line by line

`IN (1, 3, 5)`

“is one of these?” It is the readable form of a chain of `OR`s, and it scales far better: a hundred ids in `IN` is one clean statement, whereas a hundred `OR`s is unreadable.

`BETWEEN 2 AND 4`

“from 2 to 4, **including both ends**”. The inclusive ends catch people out every year: `BETWEEN 1 AND 3` includes 3.

`DISTINCT`

remove duplicate *values* from the result. Note it de-duplicates the whole selected row: `SELECT DISTINCT credits, title` returns unique *combinations*, not unique credits.

`count(*)`

count the rows.

`AS total_students`

renames the output column so the header of your result is readable; a bare `count(*)` gives you a column called `count`, which looks unprofessional the moment it lands in an API response.

Now put the two together — this is the query shape the Module 3 lab used, and the

shape every “how many X per Y” report needs:

Listing 32. TYPE AT THE PROMPT — GROUP BY produces one row per group

```
SELECT credits, count(*) AS course_count
FROM courses
GROUP BY credits
ORDER BY credits;
```

`GROUP BY credits` gathers rows into one group per distinct credits value; `count(*)` counts rows **within each group**; `ORDER BY credits` sorts the groups. Read it as: *for each credits value, how many courses have it?* This differs from the plain `SELECT count(*) FROM students` above in one crucial way — an aggregate with no `GROUP BY` collapses the entire table to **one** row, while `GROUP BY` produces one row per group. Confusing those two is the classic beginner error, so when you write it, say the sentence out loud.

Part 4 — `UPDATE`: changing existing records

TYPE AT THE PROMPT, inside a transaction

Paste the whole block. The `ROLLBACK` undoes the change.

Listing 33. TYPE AT THE PROMPT — update one course, then undo it

```
BEGIN;
UPDATE courses SET course_fee = 55000.00 WHERE id = 1;
SELECT id, title, course_fee FROM courses WHERE id = 1;
ROLLBACK;
```

Line by line

`UPDATE courses`

change rows in this table.

`SET course_fee = 55000.00`

the assignment: which column gets which new value. Comma-separate for several columns: `SET title = 'Advanced Programming', credits = 4`.

`WHERE id = 1`

the clause that decides the blast radius. One row matches, one row changes.

`ROLLBACK;`

undo it.

Two behaviours of `UPDATE` that surprise people, and both matter:

It is set-based, not row-based. `UPDATE` does not take a list of rows; it takes a *condition*, and changes **every** row that satisfies it. If your `WHERE` matches five rows, five rows change — silently. You will not get "5 rows updated" as an error; some clients report it in a status line, but the statement itself succeeds. This is why a careless `UPDATE` is more dangerous than a careless `INSERT`: a bad `INSERT` fails a constraint and damages nothing, while a bad `UPDATE` succeeds and overwrites good data.

`WHERE` is not optional in practice. This is the single most important sentence in the module:

Listing 34. NO WHERE CLAUSE — updates EVERY row in the table

```
UPDATE courses SET course_fee = 0;
```

Read that again

> No `WHERE` means no filter, which means *every row in the table*. There is no confirmation prompt, no "are you sure?", no undo. It commits immediately unless you are inside a transaction. One keystroke, and every course in your database is free.

And the version that fools people, because it *looks* targeted:

Listing 35. LOOKS targeted, IS NOT — deletes by a business rule, not by id

```
UPDATE courses SET course_fee = 0 WHERE credits = 3;
```

That is not a mistake by accident — it is a deliberate rule with a real blast radius. `credits = 3` matches every 3-credit course, and they all become free. **A `WHERE` clause that looks specific is not the same as a `WHERE` clause that is specific.** The safe pattern is to *look before you leap*: run the filter as a `SELECT` first, count the rows, and only then run the `UPDATE`.

Listing 36. The two-step habit: look first, then change

```
-- 1. LOOK: how many rows would this touch?
SELECT id, title, credits, course_fee FROM courses WHERE credits = 3;
-- 2. Only if that list is what you expected, CHANGE them:
UPDATE courses SET course_fee = 0 WHERE credits = 3;
```

Make that two-step habit explicit. It costs one extra query and it is the difference between a professional and an amateur.

Constraints still apply. `UPDATE` is a write, so every Module 3 rule is evaluated. Setting a course's credits to `-5` fails on `ck_courses_credits`; setting two courses to the same `course_code` fails on `uq_courses_code`; setting an enrollment's `student_id` to `999` fails on `fk_enrollments_student`. This is the practical payoff of Module 3: an `UPDATE` that would corrupt the data is refused by the database, not discovered by a user three weeks later.

Part 5 — `DELETE`: removing records

TYPE AT THE PROMPT, inside a transaction

Paste the whole block.

Listing 37. TYPE AT THE PROMPT — remove enrollments, then undo

```
BEGIN;
DELETE FROM enrollments WHERE student_id = 3;
SELECT count(*) FROM enrollments WHERE student_id = 3;
ROLLBACK;
```

Line by line

`DELETE FROM enrollments`

remove rows from this table. There is **no `SET` and no column list** — `DELETE` removes whole rows, it cannot blank out a single column.

If you want `course_fee` to be empty rather than the row gone, that is an `UPDATE`: `UPDATE courses SET course_fee = NULL`. Choosing between them is a real decision, not a syntax preference — deleting a row loses every column in it, while nulling one column keeps the rest.

`WHERE student_id = 3`

the blast radius, as always.

`ROLLBACK;`

undo it.

Now the two dangerous forms:

Listing 38. The two dangerous DELETE shapes

```
-- NO WHERE: deletes EVERY row in the table
DELETE FROM enrollments;

-- LOOKS targeted, ISN'T: every enrollment for any of three students
DELETE FROM enrollments WHERE student_id IN (3, 4, 5);
```

The second is the one to dwell on. It reads like it is cleaning up three students; it is deleting **every enrollment those students ever had**, across every course, irrevocably. Enrollments carry `enrolled_on` — the record of when someone joined a course — and that history is gone. Again: run the `SELECT` first, look at the rows, count them, and only then delete.

One more asymmetry worth teaching, because it is not intuitive. `DELETE FROM students WHERE id = 999;` — deleting a student who does not exist — **succeeds.** It matches zero rows, changes nothing, and reports no error. There is no "you deleted

a nonexistent thing" complaint, because there is nothing to complain about. You must check `RETURNING` to know whether your delete did anything:

Listing 39. TYPE AT THE PROMPT — RETURNING reveals that nothing was deleted

```
DELETE FROM students WHERE id = 999 RETURNING id, full_name;
```

An empty result means nothing was deleted. This is exactly the difference between SQL and an API, and Part 7 develops it — it is the reason an API endpoint must translate "zero rows affected" into a **404**, because SQL will not do it for you.

Part 6 — The consequences of incorrect updates and deletions

The roadmap asks for this explicitly, so it gets its own part. Every item below is a real failure mode with its own defence.

Transactions are the seatbelt — and they are not optional

Module 3 defined a transaction: `BEGIN;` opens it, `COMMIT;` makes it permanent, `ROLLBACK;` undoes everything since the `BEGIN`. The rule for this module, stated flatly:

The rule

> **Every `UPDATE` and every `DELETE` on data you care about runs inside a transaction. You write the `ROLLBACK` before you run the `COMMIT`, and you only type `COMMIT` once you have looked at the result.**

Listing 40. The correct sequence: write, look inside the transaction, then decide

```
BEGIN;
UPDATE students SET email = 'ada.new@school.edu' WHERE id = 1;
SELECT id, full_name, email FROM students WHERE id = 1;   -- LOOK at it
-- still inside the transaction; nothing is permanent yet
COMMIT; -- or ROLLBACK;
```

The reason the `SELECT` goes **inside** the transaction is subtle but important: it lets you see the effect of your change in the same context in which you decide whether to keep it. If you `SELECT` after a `COMMIT`, you are inspecting a permanent change and deciding too late.

`COMMIT` is a one-way door. There is no `UNCOMMIT`. If you delete a student, delete their enrollments through the cascade, and then `COMMIT`, the data is gone and no SQL command brings it back. Professional teams restore from backups; beginners do not have backups.

The cascade is a deletion multiplier

From Module 3: `enrollments` has `ON DELETE CASCADE` on both parents. So this single statement:

Listing 41. One DELETE, many rows gone

```
DELETE FROM students WHERE id = 3;
```

…deletes the student **and** every enrollment row referencing them — silently, in the same transaction, with no separate confirmation. One `DELETE`, many rows gone. This is correct behaviour for our business rule (an enrollment without a student is meaningless), but it means **the blast radius of a `DELETE` is not visible in the `DELETE` statement itself.** Before deleting any parent row, check what depends on it:

Listing 42. TYPE AT THE PROMPT — what goes with them?

```
SELECT count(*) FROM enrollments WHERE student_id = 3;
```

Always worth asking before a delete: what else does this row hold up?

Constraints turn some mistakes into non-mistakes

Not every bad `UPDATE` is catastrophic, because Module 3 is watching:

What you typed	What happens
`UPDATE courses SET credits = -5 WHERE id = 1`	Refused: `ck_courses_credits` — old data intact
`UPDATE courses SET course_code = 'CS102' WHERE id = 1`	Refused: `uq_courses_code` — CS102 already exists
`UPDATE students SET email = NULL WHERE id = 1`	Refused: `NOT NULL` on `email`
`UPDATE courses SET course_fee = 0 WHERE credits = 3`	**Succeeds.** Legal values, wrong business decision

That last row is the important one. Constraints protect the **shape** of the data; they cannot protect your **judgement**. No rule in the database knows that making three courses free was not the intention. That gap is covered by the two-step habit (look, then change) and by transactions — not by constraints. This is the honest boundary of everything Module 3 promised.

`RETURNING` makes the result visible

Use it as a safety device, not just a convenience. `UPDATE ... RETURNING *` shows you exactly which rows changed, *after* the change and *before* the `COMMIT`:

Listing 43. TYPE AT THE PROMPT — see what changed, before you commit

```
BEGIN;
UPDATE students SET date_of_birth = '2003-04-15' WHERE id = 1
RETURNING id, full_name, date_of_birth;
ROLLBACK;
```

If that returned more rows than you expected, you have just saved yourself. This is the professional habit — make the server show you the consequences of your write, while you still have the option to refuse them.

`TRUNCATE` is a different tool entirely

The seed script needs to clear tables before reseeding, and it uses `TRUNCATE` rather than `DELETE FROM`:

Listing 44. The reset line used by the Module 4 seed script

```
TRUNCATE TABLE enrollments, students, courses RESTART IDENTITY CASCADE;
```

Line by line

`TRUNCATE`

empties a table wholesale and is far faster than deleting row by row. Two keywords matter.

`RESTART IDENTITY`

resets the identity counters so the next insert gets id 1 again — without it your seeded students would start at id 47, and every hardcoded `WHERE id = 1` in the lab file would silently target the wrong row. That is not a hypothetical; it is the single most confusing failure a self-learner hits in this module.

`CASCADE`

also empties tables that reference the ones listed — **and this is a *different* cascade from `ON DELETE CASCADE`.**

`ON DELETE CASCADE` fires on `DELETE` row by row; `TRUNCATE …CASCADE` fires on the whole-table operation and reaches further, wiping dependent tables even if you did not list them. Know both, and know they are not interchangeable.

`TRUNCATE` needs higher privileges than `DELETE` and cannot be undone by `ROLLBACK` in the same way — which is another reason it belongs only in a build script you control, never in application code.

A summary of defences

Defence	What it protects against
Always use a `WHERE` clause	Updating/deleting every row by accident
`SELECT` first, then write	A `WHERE` that matches more rows than intended
`BEGIN` … `ROLLBACK`	Every mistake above, provided you do not `COMMIT`
`COMMIT` only after looking	Committing a change you have not inspected
`RETURNING *`	Not noticing how many rows you changed
Constraints (Module 3)	Writes that break the shape of the data
Foreign keys	Orphaned rows and dangling references
A backup	Everything else

That last row is not a joke. Every professional team has one. A transaction protects you from a mistake you *watched*; a backup protects you from the one you did not.

Part 7 — SQL CRUD vs API CRUD

The roadmap asks for this distinction, and it is the conceptual bridge between this course and the FastAPI work in Month 2. SQL and an API do the same four things — create, read, update, delete — but they are not the same interface, and the differences are where real bugs are born.

The mapping

`SQL (today)`	API operation	`HTTP method`	Statement it runs
`INSERT INTO students ...`	Create	`POST`	one `INSERT`, then `RETURNING *`
`SELECT ... WHERE id = 1`	Read one	`GET`	`SELECT ... WHERE id = $1`
`SELECT ... WHERE ... ORDER BY ... LIMIT ...`	Read many	`GET`	`SELECT ... WHERE ... ORDER BY ... LIMIT ...`
`UPDATE students SET ... WHERE id = 1`	Update	`PUT / PATCH`	`UPDATE ... WHERE id = $1 RETURNING *`
`DELETE FROM students WHERE id = 1`	Delete	`DELETE`	`DELETE ... WHERE id = $1 RETURNING id`

Read that table left to right and the shape is clear: **one HTTP request maps to one SQL statement, aimed at one resource, identified by one id in the URL.**

The five differences that matter

1 — SQL is set-based; an API request names one thing. `DELETE FROM students WHERE credits = 3;` is one statement that can affect thousands of rows. `DELETE /students/3` names exactly one student. Your API layer's job is to *prevent the SQL from ever being that broad* — the id in the URL becomes the `WHERE` clause, and the `WHERE` clause can never be omitted because the URL always supplies it. This is the strongest safety argument for the API pattern: **structure makes the mistake impossible.**

2 — SQL has no concept of "not found"; the API must supply one. This is the difference students are most surprised by, so slow down on it. `SELECT ... WHERE id = 999` returns an **empty result set**. It is not an error. PostgreSQL did exactly what it was asked and correctly reported that nothing matched. `DELETE ... WHERE id = 999` likewise succeeds and deletes nothing. But a client that sent `GET /students/999` expects either a student or an honest failure, and "here is an empty array" is neither. So your FastAPI code must check the row count itself and return **HTTP 404 Not Found** when it is zero. In SQL you never write that check; in an API you always must.

3 — SQL reports errors as codes; an API reports them as statuses. Module 3's five SQLSTATEs are raw material. Your application translates them:

SQLSTATE	What happened	HTTP status to return
23505	Duplicate — `uq_students_email`	**409 Conflict**
23503	Foreign key violation	**409 Conflict** (or 400 if the parent id is simply unknown)
23514	`CHECK` failed	**422 Unprocessable Entity**
23502	`NOT NULL` violated	**422 Unprocessable Entity**
42830	Bad `FOREIGN KEY` definition	a bug in your code — **500**, log it, fix it

That table is the seam between this course and the next one. Note the two layers doing their separate jobs: **422 comes from your validation layer** (the JSON was malformed — courtesy), **409 comes from the database** (the write was well-formed but violated a constraint — law).

4 — SQL does not know about HTTP verbs, and an API does not know about rows. The verb is the client's *intent*; the SQL statement is the server's *action*. `POST /students` means "here is a new student", and the server's job is to translate that intent into an `INSERT` — or to decide the request is a duplicate and reply 409. The verb carries no database meaning whatsoever; the mapping above is a convention your application defines.

5 — One API call can be several SQL statements, in a transaction. "Enrol this student in this course" is a real business operation: check the student exists, check the course exists, check they are not already enrolled, insert the enrollment. That is four statements that must all succeed or all fail together. If the insert fails after the checks, the checks must not have left anything behind. This is `BEGIN; …COMMIT;` around a block — the transaction you have been using all module, now used for its real purpose. In Python you will write `with conn.transaction():` rather than typing `BEGIN;`, but the semantics are identical to what you practised by hand today.

One preview, one warning

Preview: notice that every SQL statement above writes `WHERE id = $1` — a **placeholder**, not a literal. That is a parameterised query, and it is how your Python code will pass values in. Writing `WHERE id = ' + user_input + '` instead is **SQL injection**, where a crafted input changes the meaning of your statement and can expose or destroy the entire database. We meet it properly in Day 3. For now, just notice the placeholder and remember that it is not optional.

Warning, since the temptation is real: the moment you write application code, you will be generating SQL by building strings. Every habit from this module has to survive that translation. `WHERE` always present, look before you leap, transaction around every write, `RETURNING` to see what happened, 404 when nothing matched. SQL written by a program has exactly the same blast radius as SQL typed by hand — and the person who wrote the program will not be watching when it runs.

Part 8 — The build script, and the two rules

The same two rules as Module 3, and they matter more today:

1. **A build script contains no deliberate mistakes** — but note the qualification. A seed script *deletes* rows, which is destructive by nature. That is acceptable here because the script is defined by what it does: **reset the database to a known starting state, then fill it with sample data.** It is not a mistake, it is the script's job — as long as it never runs against data you care about. Say that out loud to your students, and say who the script is allowed to be run by.
2. **A build script is re-runnable.** Run it twice; the second run must be silent *and* leave the same data.

Rule 2 is why the script starts with `TRUNCATE` rather than `INSERT`: inserting the same five students twice would fail on `uq_students_email`. Truncating first makes the script idempotent in effect rather than in name. The seed script is thus a **fixed point** — run it any number of times and the database ends in exactly the same state — which is the property that lets a teammate, a CI pipeline, or you next Tuesday rebuild everything with one command.

APPEND TO `db/sql/03_crud_queries.sql`

> Create the file now if it does not exist, and paste the entire block below in one go. Then run it with `\i db/sql/03_crud_queries.sql`.

Listing 45. The complete seed and query script for Module 4

```
-- 03_crud_queries.sql
-- Day 2 / Module 4: the four CRUD statements against the real schema.
-- Run after 02_constraints.sql. Safe to run any number of times.
--
-- WARNING: this script DELETES all rows in students, courses and enrollments,
-- then reseeds them. Never point it at a database you care about.

-- ---------- reset to a known starting state ----------
-- dependents listed first; RESTART IDENTITY so ids begin at 1 again,
-- so every hardcoded id in the lab file means what you think it means.
```

```
TRUNCATE TABLE enrollments, students, courses RESTART IDENTITY CASCADE;

-- ---------- CREATE: insert new records ----------
INSERT INTO students (full_name, email, date_of_birth) VALUES
    ('Ada Okonkwo',    'ada@school.edu',     '2003-04-15'),
    ('Tunde Balogun',   'tunde@school.edu',   '2002-09-30'),
    ('Chiamaka Nwosu',  'chiamaka@school.edu','2004-01-12'),
    ('Fatima Bello',    'fatima@school.edu',  '2001-07-04'),
    ('Emeka Obi',       'emeka@school.edu',   NULL);

INSERT INTO courses (course_code, title, credits, course_fee) VALUES
    ('CS101',  'Introduction to Programming',  3, 50000.00),
    ('CS102',  'Structured Logic Design',      3, 50000.00),
    ('MTH201', 'Introductory Mathematics',     4, 45000.00),
    ('ICT110', 'Computer Literacy',           2, 35000.00),
    ('ADM101', 'Free Community Seminar',       1,     0.00);

-- enrollments: one row = one student's place in one course
INSERT INTO enrollments (student_id, course_id) VALUES
    (1, 1), (1, 2), (2, 1), (2, 3), (3, 2),
    (4, 4), (5, 1), (5, 5);

-- ---------- READ: retrieve records ----------
-- one student by primary key
SELECT id, full_name, email, date_of_birth FROM students WHERE id = 1;

-- everyone, alphabetical
SELECT id, full_name, email FROM students ORDER BY full_name;

-- students with no date of birth on file
SELECT id, full_name FROM students WHERE date_of_birth IS NULL;

-- courses a given student is taking
SELECT c.course_code, c.title, e.enrolled_on
FROM enrollments e
JOIN courses c ON c.id = e.course_id
WHERE e.student_id = 1
ORDER BY c.course_code;

-- how many students are enrolled in each course
SELECT c.course_code, count(*) AS enrolled_count
FROM enrollments e
JOIN courses c ON c.id = e.course_id
GROUP BY c.course_code
ORDER BY enrolled_count DESC, c.course_code;

-- the three most expensive courses
SELECT course_code, title, course_fee
FROM courses
ORDER BY course_fee DESC
LIMIT 3;

-- ---------- UPDATE: change existing records ----------
-- look before you leap: which courses have 3 credits?
SELECT id, course_code, credits FROM courses WHERE credits = 3;

-- then change them, inside a transaction, and roll back
BEGIN;
```

```
UPDATE courses
SET title = 'Introduction to Programming (Revised)', credits = 4
WHERE id = 1
RETURNING id, title, credits;
ROLLBACK;   -- deliberately undone: the seed data stays as written

-- ---------- DELETE: remove records ----------
-- look before you leap: how many enrollments does student 5 have?
SELECT * FROM enrollments WHERE student_id = 5;

-- remove them inside a transaction, then undo
BEGIN;
DELETE FROM enrollments WHERE student_id = 5
RETURNING student_id, course_id;
ROLLBACK;   -- deliberately undone: the seed data stays as written
```

Line by line — the parts that are new

`TRUNCATE TABLE enrollments, students, courses RESTART IDENTITY CASCADE;`

discussed fully above. The three tables are listed because `CASCADE` would reach them anyway; listing them makes the destruction explicit rather than implied, and that is a deliberate teaching choice. `RESTART IDENTITY` is not optional in this script.

`INSERT INTO students ... VALUES (5 tuples);`

The first `INSERT` uses the multi-row form: one column list, five tuples, one statement, one semicolon at the end.

`'Emeka Obi' …NULL`

has `NULL` for `date_of_birth` — deliberate, because `date_of_birth` is the one nullable column in the schema, and a seed row with a `NULL` gives you something to test `IS NULL` against in the lab. `NULL` is written as the bare word `NULL`, never quoted; `= NULL` in single quotes is the **string** "NULL", which is the most common seed-data bug there is.

`ADM101 …0.00`

has `course_fee` of `0.00` — deliberate, because `ck_courses_fee` uses `>= 0`, so a genuinely free course is legal, and having one in the data makes that design decision visible rather than theoretical.

The `JOIN` queries are a preview. **You have not met `JOIN` yet** — it is Day 3's core topic. Read them as: "from the `enrollments` rows, follow the reference to the matching `courses` row and show me the course". They are included here so you can see the shape of the questions this data answers, and so that when Day 3 arrives you already know what the output should look like. Do not worry if the syntax does not click today.

`BEGIN; …ROLLBACK;`

in the `UPDATE` and `DELETE` sections, with the `COMMIT` deliberately absent and a comment saying so. This is the pattern the whole module builds towards: the seed script *shows* you the dangerous statement running safely, and then undoes it, so the data ends in the documented state. Note that the `SELECT` before each one is the two-step habit in action — look, then change.

Part 9 — Workbook part one: the build script

1. **Create `db/sql/03_crud_queries.sql`** next to `02_constraints.sql`, and paste the whole block from Part 8.
2. **Save it** (Ctrl+S) — then run it. Save first; this is the habit, not the ceremony.
3. **Run it** from `psql`: `\i db/sql/03_crud_queries.sql`
4. **Check the counts.** You should see 5 students, 5 courses, 8 enrollments:

Listing 46. Verify the seed counts

```
SELECT (SELECT count(*) FROM students)   AS students,
       (SELECT count(*) FROM courses)    AS courses,
       (SELECT count(*) FROM enrollments) AS enrollments;
```

5. **Run it a second time.** It must succeed silently and leave the same counts. If it fails, you have not used `TRUNCATE` at the top — that is the whole reason it is there.
6. **Confirm the `UPDATE` and `DELETE` in the script were rolled back** — `SELECT title FROM courses WHERE id = 1;` must still read `Introduction to Programming`, and `SELECT count(*) FROM enrollments WHERE student_id = 5;` must still be 2.

Part 10 — The lab file

Everything in the next file is an experiment, and several of them are **designed to damage data**. Every task that writes is wrapped in `BEGIN` …`ROLLBACK`, and the rollback is written below the damage so that if the statement hangs or you get interrupted, the recovery line is already on your screen. That is the discipline this module is trying to install: **the undo is planned before the action.**

APPEND TO `db/sql/lab/04_lab_crud.sql`

Paste the whole block below.

Listing 47. The complete lab file for Module 4

```
-- 04_lab_crud.sql
-- Experiments for Day 2 / Module 4. Not part of the build.
-- How to use: uncomment ONE task, save, run \i db/sql/lab/04_lab_crud.sql,
-- read the result, then comment it back out.
-- SAFETY: every writing task is wrapped in BEGIN ... ROLLBACK.
```

```
-- If you are interrupted between BEGIN and ROLLBACK, type ROLLBACK; immediately.

-- ============================================================
-- TASK A - read one row by primary key
-- Expected: exactly one row, Ada Okonkwo
-- ============================================================
-- SELECT id, full_name, email FROM students WHERE id = 1;

-- ============================================================
-- TASK B - SELECT * versus naming columns
-- Expected: identical data here; notice which one survives an ALTER TABLE later
-- ============================================================
-- SELECT * FROM courses WHERE id = 1;

-- ============================================================
-- TASK C - NULL needs IS NULL, never = NULL
-- Expected: first query returns Emeka Obi; second returns NOTHING at all,
--           with no error. That silence is the lesson.
-- ============================================================
-- SELECT id, full_name FROM students WHERE date_of_birth IS NULL;
-- SELECT id, full_name FROM students WHERE date_of_birth = NULL;

-- ============================================================
-- TASK D - the NOT / NULL trap
-- Expected: the second query is missing Emeka Obi, and you did not notice
-- ============================================================
-- SELECT id, full_name FROM students WHERE id <> 3;
-- SELECT id, full_name FROM students WHERE id <> 3 OR date_of_birth IS NULL;

-- ============================================================
-- TASK E - ORDER BY then LIMIT; the three newest students
-- Expected: 3 rows, most recent first
-- ============================================================
-- SELECT id, full_name, created_at FROM students
-- ORDER BY created_at DESC LIMIT 3;

-- ============================================================
-- TASK F - LIKE wildcards: % is "anything", _ is "one character"
-- Expected: %@school.edu matches everyone; a_a matches a 3-letter name
-- ============================================================
-- SELECT id, email FROM students WHERE email LIKE '%@school.edu';
-- SELECT id, full_name FROM students WHERE full_name LIKE 'a_a';

-- ============================================================
-- TASK G - THE DANGEROUS ONE: UPDATE with no WHERE clause
-- Expected: every course fee becomes 0. ROLLBACK undoes it.
--           Run it, then run the SELECT below to prove the rollback worked.
-- ============================================================
-- BEGIN;
-- UPDATE courses SET course_fee = 0;                  -- NO WHERE. ALL ROWS.
-- SELECT course_code, course_fee FROM courses ORDER BY course_code;
-- ROLLBACK;
-- SELECT course_code, course_fee FROM courses ORDER BY course_code;

-- ============================================================
-- TASK H - a WHERE clause that LOOKS targeted and is not
-- Expected: CS101 and CS102 both drop to 0 fee, not just one
-- ============================================================
```

```
-- BEGIN;
-- SELECT id, course_code, credits FROM courses WHERE credits = 3;  -- LOOK first
-- UPDATE courses SET course_fee = 0 WHERE credits = 3;
-- SELECT course_code, course_fee FROM courses WHERE credits = 3;
-- ROLLBACK;

-- ============================================================
-- TASK I - RETURNING shows you exactly what you changed
-- Expected: one row back, showing the NEW value, before you commit
-- ============================================================
-- BEGIN;
-- UPDATE students SET email = 'ada.new@school.edu' WHERE id = 1
-- RETURNING id, full_name, email;
-- ROLLBACK;

-- ============================================================
-- TASK J - constraints still guard an UPDATE
-- Expected: ERROR ... violates check constraint "ck_courses_credits"
--           and the old value is untouched
-- ============================================================
-- BEGIN;
-- UPDATE courses SET credits = -5 WHERE id = 1;
-- ROLLBACK;
-- SELECT credits FROM courses WHERE id = 1;

-- ============================================================
-- TASK K - deleting something that does not exist SUCCEEDS
-- Expected: no error, no rows. Now try it with RETURNING - still empty.
--           This is why an API endpoint must answer 404 itself.
-- ============================================================
-- DELETE FROM students WHERE id = 999;
-- DELETE FROM students WHERE id = 999 RETURNING id, full_name;

-- ============================================================
-- TASK L - the cascade is a deletion multiplier
-- Expected: 2 enrollments before, 0 after the DELETE of ONE student,
--           then 2 again after ROLLBACK
-- ============================================================
-- BEGIN;
-- SELECT count(*) FROM enrollments WHERE student_id = 5;   -- 2
-- DELETE FROM students WHERE id = 5;                       -- takes 2 with it
-- SELECT count(*) FROM enrollments WHERE student_id = 5;   -- 0
-- ROLLBACK;
-- SELECT count(*) FROM enrollments WHERE student_id = 5;   -- 2 again

-- ============================================================
-- TASK M - aggregates: GROUP BY produces one row per group,
--           no GROUP BY collapses the whole table to one row
-- Expected: a small breakdown of credits versus course count
-- ============================================================
-- SELECT credits, count(*) AS course_count FROM courses
-- GROUP BY credits ORDER BY credits;
-- SELECT count(*) AS all_courses FROM courses;   -- one number, not a group

-- ============================================================
-- TASK N - reset whenever the data looks wrong
-- Expected: the original 5 / 5 / 8 counts, ids back at 1
-- ============================================================
```

```
-- \i db/sql/03_crud_queries.sql
```

Line by line, for the parts that are new

Every task header states its **expected output** — read it before running, so you know what success looks like.

Task C is the pair that teaches the `NULL` trap permanently: the second query returns nothing and reports no error, which is the whole point.

Task D shows the subtler cousin of the same bug, where rows go missing without a `NULL` comparison in sight.

Tasks G and H are the module's centrepiece — Task G has no `WHERE` at all, Task H has one that looks specific and is not; between them they cover both ways people actually lose a table.

Task K is the quiet one: deleting a row that does not exist is not an error, and that fact is the direct justification for HTTP 404 in Part 7. Task L shows a single `DELETE` removing three rows in total.

Task N is your escape hatch — whenever the data looks wrong, re-run the seed script and start clean. Get into the habit of using it rather than trying to repair data by hand.

Part 11 — Workbook part two: verification checklist

#	Check	Command or task	Expected
1	Seed counts are right	The Part 9 count query	5 students, 5 courses, 8 enrollments
2	Ids restart at 1	`SELECT id, full_name FROM students ORDER BY id;`	1 to 5, contiguous
3	The script is re-runnable	Run `03_crud_queries.sql` twice	Silent second run, same counts
4	`NULL` is present and findable	Lab Task C	Emeka Obi; the `= NULL` query returns nothing
5	Free course is legal	`SELECT * FROM courses WHERE course_fee = 0;`	ADM101 only
6	Constraint still guards updates	Lab Task J	Error on `ck_courses_credits`; old value intact
7	`RETURNING` works	Lab Task I	One row with the new value
8	Deleting nothing is not an error	Lab Task K	No error, empty `RETURNING`
9	Cascade is understood	Lab Task L	$2 \to 0 \to 2$ across the transaction
10	Rollback works	After any lab task	Data identical to the seed state
11	No transaction left open	Check for warnings at the prompt	If psql warns about an open transaction, type `ROLLBACK;`

Part 12 — Troubleshooting

What you see	Why	What to do
`ERROR: current transaction is aborted, commands ignored until end of transaction block`	One statement in your transaction failed; PostgreSQL refuses every subsequent statement until you end the block	Type `ROLLBACK;`, fix the cause, start the transaction again
`ERROR: syntax error at or near "NULL"`	You wrote `= NULL` or quoted it as `'NULL'`	Use `IS NULL` / `IS NOT NULL`; write `NULL` bare, unquoted
`ERROR: relation "students" does not exist`	Wrong database, or you are not in the project folder	Check the prompt says `school_db`; re-check the path in `\i`
`could not open file "..."`	`\i` paths are relative to where you launched `psql`, not where the file is	`cd` to the project root first, or use an absolute path; forward slashes on Windows
`ERROR: duplicate key value violates unique constraint "uq_students_email"`	You inserted a seed row twice — usually because `TRUNCATE` is missing or commented out	Run the seed script; it truncates first
`A SELECT returns nothing when rows should match`	Almost always `= NULL`, or a `WHERE` clause with the wrong column or case	Re-run the query as `SELECT * FROM t WHERE <same condition>;` and widen one step at a time
`Everything vanished after a lab task`	A task ran without its `ROLLBACK`, or you typed `COMMIT`	Re-run `\i db/sql/03_crud_queries.sql` to reset — Task N exists for this
`Seeded ids are not 1 to 5`	`RESTART IDENTITY` missing from the `TRUNCATE`	Add it, or re-run the seed script

Part 13 — Instructor notes

Timing (140 minutes). Parts 0–2 (anchor, DML/CRUD, `INSERT`) ≈ 25 min. Part 3 (`SELECT`) ≈ 40 min — the largest topic, do not compress it; the `NULL` material alone justifies 10 minutes. Parts 4–5 (`UPDATE`, `DELETE`) ≈ 20 min. Part 6 (consequences, transactions, cascade) ≈ 25 min — the intellectual core; protect its time. Part 7 (SQL vs API CRUD) ≈ 15 min. Parts 9–12 (workbook, verification, troubleshooting) ≈ 15 min.

Live-demo order. Run Task G in front of the class, inside a transaction, and let them watch every course fee become zero. Then `ROLLBACK` and watch it all come back. Nothing else in the module makes the point as viscerally — the fear of forgetting a `WHERE` clause is what keeps the habit alive afterwards. Follow it with Task H, the subtler version, because that is the mistake they will actually make.

The moment to stop and slow down. Lab Task K. “Deleting a row that does not exist succeeds and reports nothing” is counter-intuitive, and it is the direct setup for Part 7.2’s HTTP 404 discussion. Connect them explicitly: *SQL will never tell your API that a student does not exist, so your API code has to check and answer 404 itself.* Students who grasp this do not write broken endpoints later.

Misconceptions to expect, and the one-line answer.

- *"A `WHERE` clause makes an `UPDATE` safe."* — It limits the damage; it does not remove it. Task H is a `WHERE` clause that was not specific enough. Look first, then write.
- *"`ROLLBACK` will get my data back after I commit."* — No. `COMMIT` is one-way; only a backup survives past it.
- *"`DELETE` is the opposite of `INSERT`."* — Not in a database. `INSERT` adds alongside; `UPDATE` changes in place; `DELETE` removes and leaves gaps.
- *"Deleting something that isn't there should error."* — It does not. SQL reports "zero rows matched", which is a success, and that is why APIs must answer 404 themselves.
- *"`= NULL` should work like `= 0`."* — No. Comparing to `NULL` yields unknown, never true, so it matches nothing, silently.
- *"`SELECT *` is fine, it's shorter."* — Fine in `psql`, not in application code: it breaks silently when someone adds a column.
- *"Constraints stop bad updates."* — They stop bad *shapes*. Task H is legal data and the wrong decision; no constraint can catch that.
- *"One SQL statement can update many rows, so the API can too."* — The API names one resource per request; the id in the URL *is* the `WHERE`. That restriction is a safety feature.
- *"`SELECT` is DML like the others."* — Often grouped that way, but `SELECT` is the only one that cannot destroy data by accident. That asymmetry is the safety story.

Discussion, if time allows (5 min). *"Should deleting a student also delete their enrollments, or should it be refused?"* Our answer is `ON DELETE CASCADE` on the grounds that an enrollment without a student is a dangling sentence — but `NO ACTION` (refuse the delete) is a legitimate choice many real systems make, because enrolment history is often a compliance record. Neither is wrong; the point is that the choice belongs to the business, and the database is the place where it gets recorded and enforced.

Preview only, one sentence each, do not open today: `UPDATE ... FROM` and subqueries (writing rows from a query, needed for realistic bulk changes); `DELETE` with `USING`; `ON CONFLICT` / upserts (insert-or-update in one statement, which is exactly what an API's "create or replace" endpoint needs). Each of these is a natural extension of what was practised today.

Version note. PostgreSQL 18 throughout. The `ON CONFLICT` syntax changed slightly in 17 (`ON CONSTRAINT` support) — previewed above, not used today, so nothing in this module is version-sensitive beyond what Modules 1–3 already established.

Part 14 — Submission

1. **`db/sql/03_crud_queries.sql`** — committed, runs silently twice, produces 5 / 5 / 8 rows. Include the output of the count query from Part 9.
2. **`db/sql/lab/04_lab_crud.sql`** — with tasks G, H, I, K and L uncommented at least once each (submit the file re-commented and tidy).
3. **A results log** for tasks C, D, G, K and L — screenshots or copied terminal blocks. For **G** and **L** in particular, include the output *before*, *inside*, and *after*

the transaction, so the rollback is visible in the evidence.

4. **A written trace of Task H** — which rows the `SELECT` showed before the `UPDATE`, which rows changed, and what the difference teaches about `WHERE` clauses.
5. **Five written answers, one or two sentences each:** (a) why `= NULL` returns nothing; (b) why deleting a nonexistent row is not an error, and what an API must return instead; (c) why `TRUNCATE` needs `RESTART IDENTITY` for this script to be trustworthy; (d) which of your five courses would a `WHERE credits = 3` update have hit in this seed data, and why; (e) which SQLSTATEs from Module 3 you expect to see in your FastAPI logs first, and what HTTP status each becomes.
6. **A one-paragraph reflection:** name the single habit from Part 6 you will keep in your own code, and why that one.

Part 15 — Where this goes next

The database is now populated, queryable, and — crucially — *safe to write to*. `students`, `courses` and `enrollments` hold realistic data; the constraints from Module 3 guard every write; and the consequences of a careless `UPDATE` or `DELETE` have been demonstrated rather than merely described.

Day 3 takes the two skills that remain. `JOIN` turns these three tables into real questions — which students are in which courses, which courses nobody has taken, which fees are actually being paid — which is the `JOIN` you previewed in the seed script. And transactions, roles and privileges, parameterised queries and the FastAPI architecture diagram turn all of today into an API. The mapping table in Part 7 is your bridge: every SQL statement you wrote by hand today becomes a statement your Python code will send, with a placeholder where the value goes, inside a transaction, with the errors translated to status codes.

The final deliverable of Day 3 is a **documented backend architecture and data-flow plan** for the Student Course Enrollment API — which is exactly what Modules 3 and 4 made possible.

Sources

Module 3 — PostgreSQL 18 documentation

- Constraints — https://www.postgresql.org/docs/18/ddl-constraints.html
- Unique constraints — https://www.postgresql.org/docs/18/ddl-constraints.html#DDL-CONSTRAINTS-UNIQUE-CONSTRAINTS
- Check constraints — https://www.postgresql.org/docs/18/ddl-constraints.html#DDL-CONSTRAINTS-CHECK-CONSTRAINTS
- Foreign key constraints and `ON DELETE` actions — https://www.postgresql.org/docs/18/ddl-constraints.html#DDL-CONSTRAINTS-FOREIGN-KEYS
- `ALTER TABLE` (adding and dropping constraints) — https://www.postgresql.org/docs/18/sql-altertable.html
- `CREATE TABLE` — https://www.postgresql.org/docs/18/sql-createtable.html
- `psql` meta-commands (`\i`, `\d`, `\dt`) — https://www.postgresql.org/docs/18/

app-psql.html

Module 4 — PostgreSQL 18 documentation

- `INSERT` — https://www.postgresql.org/docs/18/sql-insert.html
- `SELECT` — https://www.postgresql.org/docs/18/sql-select.html
- `UPDATE` — https://www.postgresql.org/docs/18/sql-update.html
- `DELETE` — https://www.postgresql.org/docs/18/sql-delete.html
- `TRUNCATE` — https://www.postgresql.org/docs/18/sql-truncate.html
- `RETURNING` — https://www.postgresql.org/docs/18/dml-returning.html
- Transactions and `BEGIN`/`COMMIT`/`ROLLBACK` — https://www.postgresql.org/docs/18/tutorial-transactions.html
- `LIKE` and pattern matching — https://www.postgresql.org/docs/18/functions-matching.html
- `NULL` handling and comparison semantics — https://www.postgresql.org/docs/18/functions-comparison.html
- Aggregates, `GROUP BY` and `HAVING` — https://www.postgresql.org/docs/18/tutorial-sql-commands.html
- Errors (`SQLSTATE`) reference — https://www.postgresql.org/docs/18/errcodes-appendix.html

A note on this document

Both modules follow the same contract. Every listing names the file it belongs in and how to run it; build scripts are kept separate from lab scripts so that a build script can always be re-run cleanly; and every concept is defined before it is used. The modules assume Modules 1 and 2, and Module 4 assumes Module 3's constraints are in place.